

Linear Regression Models on House Price of King County

CSCI E-106 Final Project Fall 2023

Jiang Lu, Tanate Locurlert, Wenjun Liu, Seema Kavatkar, Hina Imran, Savannah Witham

18 December 2023

Abstract

This is a group project report of course CSCI-106 Fall 2023. The members of the group are shown on the author list. We focus on statistical modeling to predict house price based on the house sale data of King County, Washington between 2014 - 2015. We build a variety of parametric and non-parametric models and choose Neural Networks as our champion model, LASSO and multiple linear regression model as the benchmark models.

The limitation of the models we presented is that the data was restricted to basic features of houses. The socia-economical status and geographic features were not included in the models.

Contents

I. Introduction	2
I. Description of the data and quality	4
III. Model Development Process	7
IV. Model Performance Testing	9
V. Challenger Models (15 points)	28
VI. Model Limitation and Assumptions	34
VII. Ongoing Model Monitoring Plan	35
VIII. Conclusion	36
Bibliography	36
Appendix	36

Executive Summary

Objective:

The goal of this data project is to develop a predictive model for housing prices using a dataset containing various features related to home sales. Our primary focus is to build a regression model to predict house price effectively.

Data Overview:

The dataset encompasses diverse variables providing insights into home characteristics, including the number of bedrooms and bathrooms, square footage, location, condition, and grade, among others. The target variable, “price,” represents the sale price of the homes for the Washington state King County between beginning of May 2014 to end of May 2015.

Key Predictors:

Essential metrics for assessing the size and functionality of a home include number of bedrooms, number of bathrooms, footage of livable area, and lot size. Property’s features and desirability are presented by waterfront and a good view of surroundings. Condition and grade are metrics that reflect the overall condition and quality of the home. number of years since built and number of years since renovated are factors that indicate the age and renovation history of the property. Location-based features are average lot size of the nearest 15 houses and average footage of the nearest 15 houses.

Approaches:

We create and test different regression models on both train and test data sets. Neural Networks gives best results among all models, followed by LASSO and multiple linear regression. Diagnostics and remedial measures are conducted thoroughly for every models we try. Most of the R packages and knowledge we gain from the present course CSCI-E106. Additional reference websites are GeeksforGeeks (GeeksforGeeks 2008), Wikipedia etc.

Conclusion:

This data project aims to build an accurate model for predicting housing prices. The results obtained will not only contribute to a better understanding of the factors influencing home prices but also offer practical guidance for individuals involved in the real estate market.

I. Introduction

A brief literature review

Before we dive into the data analysis of house sales prices in King County, Washington, we look into a couple of review papers. The first is a survey on methods and input data types (Geerts, M.; vanden Broucke, S.; De Weerd, J. 2023) that summarizes ninety-three publications on modeling methods and types of input data. The methods range from basic techniques such as multiple regression analysis or spatial econometric models, to time series, fuzzy logic, decision trees, and also novel methods like artificial neural networks and deep learning. Input data categories vary as well. Data of standard features includes structural, temporal, socioeconomic, and environment. Then, there is standard and advanced level spatial data. Some publications use graphs and other unstructured data such as image and text.

We also review an assessment (Chwiałkowski, C.; Zydron, A.; Kayzer, D. 2022) on factors that affect dwelling price which summarizes the categories of attributes that highly related with house price based on literature review (MIRANDA CRACE 2023). It draws our attention to not only the physical characteristics, but also the characteristics of neighborhood, environment, and economic conditions.

Focus of present analysis

Although there are many factors associated with a house’s price and there could be countless methods to be applied to house valuation and price prediction, the present report focuses on building linear regression models to predict sales prices of real estate properties using the house sales data collected in King County, Washington, USA between the years 2014 and 2015. Since the data we use for modeling is limited to certain physical characteristics and neighborhood comparison, the interpretation of estimates, statistics and prediction is bound to these physical characteristics and neighborhood data.

We divide the whole data set into a random training data set accounting for 70% of the raw data set and the remaining is used as a test data set to test the performance of the models we build here. We start with exploratory data analysis (EDA) to summarize the uni-variate distribution and bi-variate correlation. EDA helps us seek the potential remedial measures. Following EDA, we conduct multiple linear regression on selected predictors to build a full model which is the starting point of our modeling process. With the residuals and fitted values available from the full model, we are able to carry out the model diagnostics, then the remedial measures, such as transformation on response and predictors, detecting influential data points and checking heteroscedasticity. Following diagnostics, we conduct variable selections, such as best subset and stepwise selection to identify significant predictors. We also try other methods such as the weighted least square (WLS), ridge, lasso,

elasticNet, iterated re-weighted least square regression, etc. At last, we transform the house price from continuous type to a binary variable so that we could build a logistic regression model to work around the normality assumptions of linear (Gaussian) regression.

I. Description of the data and quality

Looking into the raw data of house sold price in King County, Washington, USA, we carry out the following steps to transform and understand the raw data better.

1. We extract strings and transform several key variables into the format that is more appropriate for analysis.
2. We then detect an algebraic linear relation among `sqft_living = sqft_above + sqft_basement`. In this case, we may only include two out of the three variables that in the linear regression model to avoid singularity issue when build a multiple linear regression model.
3. Reminded by Jussan's post and Rafael's response on Ed Discussion, we use min-max normalization method to scale the house price to between 0 and 1.

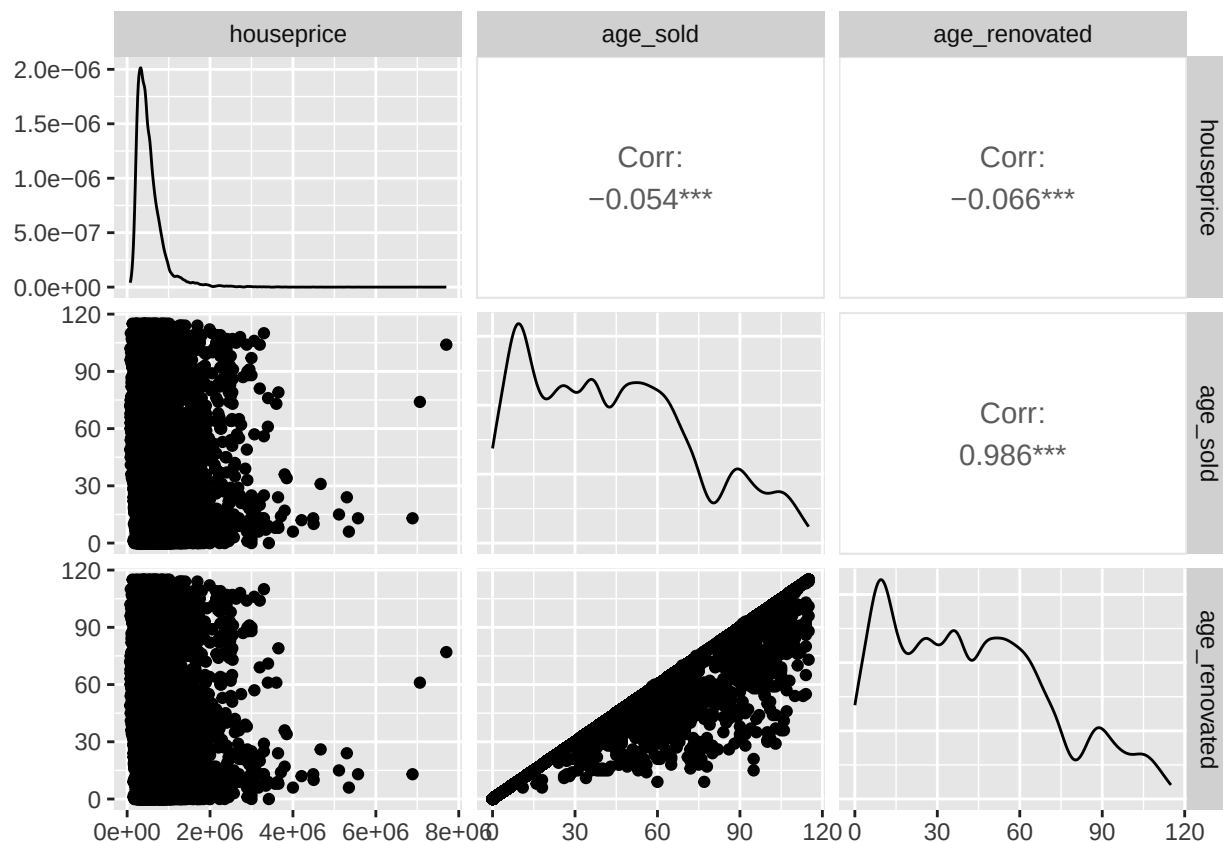
Key variables are listed below including `houseprice_normlz`, the response variable and 13 predictors described below. We temporarily exclude zip code, latitude, and longitude from key predictors due to the complexity of interpretation of these predictors in present modeling. These excluded variables might be useful when we explore predictors of spatial and socioeconomic status, etc. We will discuss the potential to include these three variables in later sections.

Key Variable	Description	Type
<code>houseprice</code>	<i>House sold price in US dollar</i>	numeric
<code>houseprice_normlz</code>	<i>House sold price normalized by min-max feature scaling</i>	numeric
<code>age_sold</code>	<i>The number of years between house sold and built</i>	numeric
<code>age_renovated</code>	<i>The number of years between house sold and renovated, or built if no renovation</i>	numeric
<code>bedrooms</code>	<i>Number of bedrooms</i>	integer
<code>bathrooms</code>	<i>Number of bathrooms, where "5" accounts for a bathroom with a toilet but no shower</i>	numeric
<code>sqft_lot</code>	<i>Square footage of the land space</i>	integer
<code>sqft_above</code>	<i>The square footage of the interior housing space that is above ground level</i>	integer
<code>sqft_basement</code>	<i>The square footage of the interior housing space that is below ground level</i>	integer
<code>grade</code>	<i>An index from 1 to 13, where 1-3 falls short of building construction and design, 7 has an average level of construction and design, and 11-13 has a high-quality level of construction and design.</i>	integer
<code>condition</code>	<i>An index from 1 to 5 on the condition of the apartment,</i>	integer
<code>waterfront</code>	<i>A dummy variable for whether the apartment was overlooking the waterfront or not</i>	integer
<code>view</code>	<i>An index from 0 to 4 of how good the view of the property was</i>	integer
<code>floors</code>	<i>Number of floors</i>	numeric
<code>sqft_living15</code>	<i>The square footage of interior housing living space for the nearest 15 neighbors</i>	integer
<code>sqft_lot15</code>	<i>The square footage of the land lots of the nearest 15 neighbors</i>	integer

The first step of data analysis is exploratory data analysis (EDA) including descriptive data analysis at uni-variate and bi-variate level. A summary table of descriptive statistics of key variables is shown in Appendix.

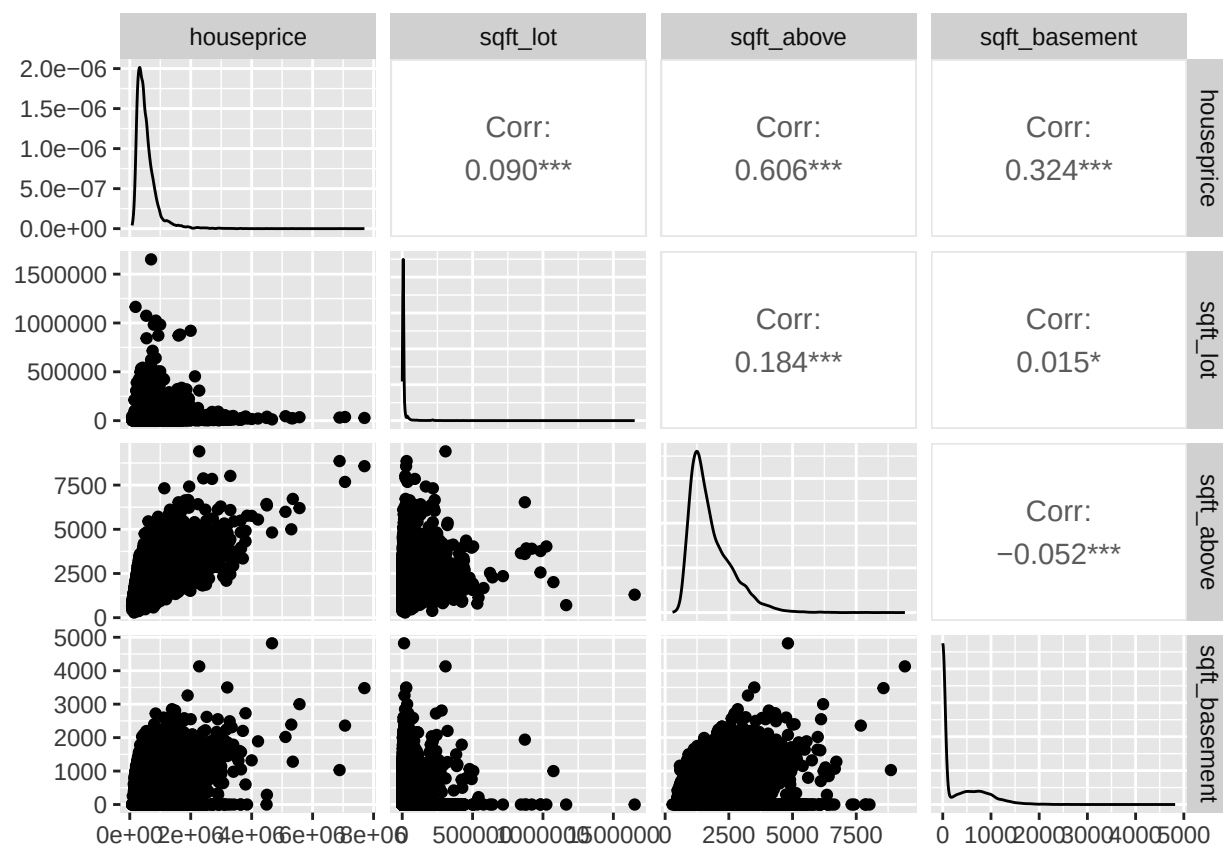
The distribution of the house prices is highly skewed with long tail to positive side. It is not normally distributed. Plots below show key variables in pairs to examine the bivariate relations by category.

```
GGally::ggpairs(hs_keyvars, columns = c(1, 3:4))
```



The house price has significant correlation with all three size measures related with a house. The footage above ground has the highest correlation with house sale price.(coefficient = 0.606

```
GGally::ggpairs(hs_keyvars, columns = c(1,5:7))
```

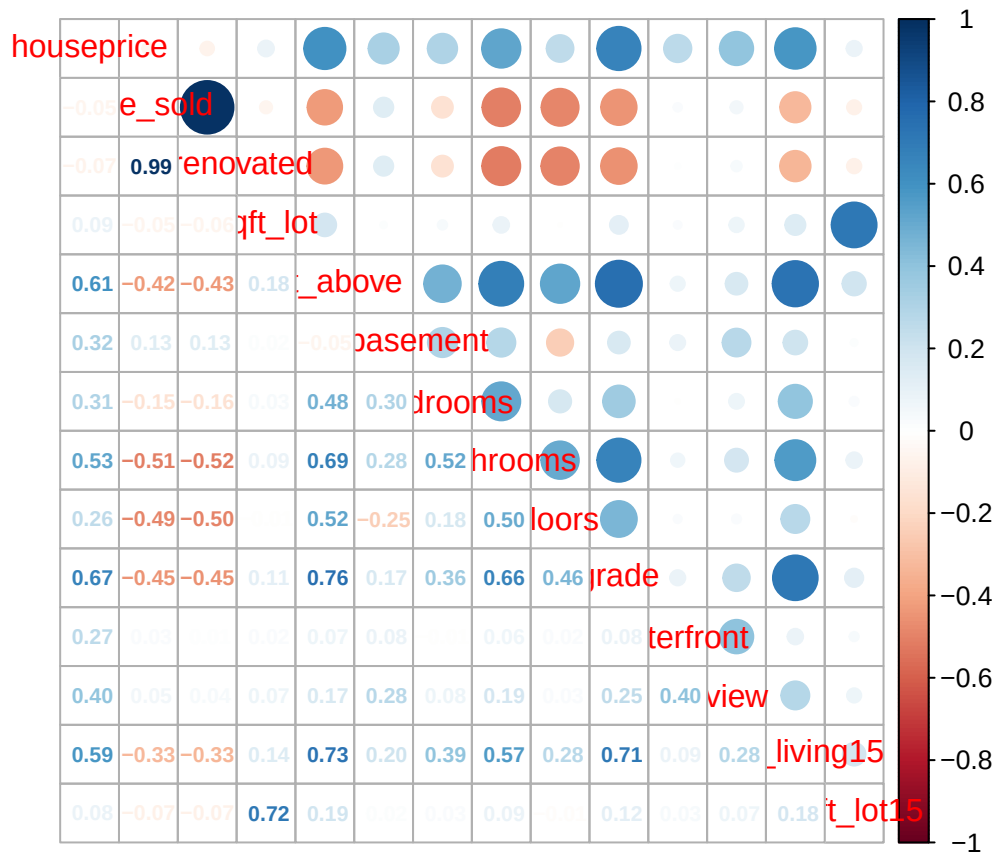


The number of bedrooms, bathrooms and floors all correlate with house price significantly. Grade and condition are two

factors that indicate the quality of the house. The correlation between house price and grade is as high as 0.667. The scatter plots for all variables can be found in Appendix.

The average footage of living area combined with the lot size of the nearest fifteen neighbors' houses is neighborhood data that might be a good predictor of house price. The correlation of the footage of living area and sale price is correlated with coefficient of correlation 0.585. The number of years passed between house built and house sold (`age_sold`) has extremely high correlation with the number of years since renovation (`age_renovated`). The coefficient of correlation is as high as 0.986, see correlation table below.

```
corr_hskeys <- cor(hs_keyvars[, -c(2,12)])
corrplot.mixed(corr_hskeys, number.cex= 0.7)
```



III. Model Development Process

We start with splitting the whole data set into a training set and a testing set. Then, we build a multiple linear model with all the predictors that we deem as key variables.

```
#creating dummy cols
d1_dummies<-dummy_cols(hs_keyvars, select_columns = 'view',remove_first_dummy = TRUE)
d2_dummies<-dummy_cols(d1_dummies, select_columns = 'waterfront', remove_first_dummy = TRUE)
d3_dummies<-dummy_cols(d2_dummies, select_columns = 'condition',remove_first_dummy = TRUE)
d3_dummies<-subset(d3_dummies, select = -c(houseprice_normlz,condition,view,waterfront))

#divide data in test and train datasets with dummy variables
set.seed(1023)
n<-dim(d3_dummies)[1]
IND=sample(c(1:n),n*0.7)
hstrain_noscale = d3_dummies[IND,]
hstest_noscale = d3_dummies[-IND,]
```

Before we build the full model of multiple linear regression, we can view the following as a summary of data transformation and selection from the raw data set.

1. Transform or convert
 - a. Convert the original string type of **price** to numeric value and further scaled (min-max normalization) to **houseprice_normlz** with a range between 0 and 1.
 - b. Extract the string from **date** and convert to **date_sold** in conventional date format.
 - c. Extract the information of year from **date_sold**.
2. Create two new predictors
 - a. **age_sold**: The number of years between house sold and built
 - b. **age_renovated**: The number of years between house sold and renovated (or built if no renovation)
3. Exclude 7 variables from predictors
 - a. **id**
 - b. **date**
 - c. **yr_built**
 - d. **yr_renovated**
 - e. **zipcode**
 - f. **lat**
 - g. **long**
 - h. **age_renovated** : due to high correlation with **age_sold**
4. Examine possible categorical variables
 - a. **waterfront** originally takes values 0 or 1 which is a dummy variable
 - b. **grade** with a numeric range from 1 to 13 indicates the quality of construction and design. We treat it as numeric predictor.
 - c. **condition** and **view** are both index variables with the ranking range between 1-5 and 1-4, respectively. We create dummy variables for those to treat these two index.

First order linear model

In the full model of multiple linear regression, there are interesting observations on the change of house sold price associated with different variables.

1. Holding all other predictors constant, the lot size has a negative linear relation with house price such that one square foot increase in lot size is associated with a \$4064 decrease in house price.
2. Holding all other predictors constant, the number of bedrooms in a house has a negative linear relation with house price such that one more bedroom is associated with a \$37130 decrease in house price.
3. Holding all other predictors constant, the nearest 15 neighbors' lot size has a negative linear relation with house price such that one more square feet of lot size is associated with a \$44 decrease in house price.
4. The **waterfront** feature seems has the most magnitude of positive relation with the house price. Holding other predictors constant, with **waterfront** feature, the house price increases by \$528,400.
5. The **grade** has the second most magnitude of positive relation with the house price. Holding other predictors constant, with each 1 unit increase in **grade**, the house price increases by \$118,700.

6. Model is significant with p-value: $< 2.2e-16$, Multiple R-squared=0.6531 and Adjusted R-squared=0.6528. All variables are significant except for 'sqft_lot' and 'age_renovated'.

```
first_model<-lm(houseprice~., data=hstrain_noscale)
summary(first_model)

##
## Call:
## lm(formula = houseprice ~ ., data = hstrain_noscale)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1259622  -108733   -10138    89336   4399130
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  -1.004e+06  4.843e+04 -20.739  < 2e-16 ***
## age_sold      2.982e+03  3.415e+02   8.730  < 2e-16 ***
## age_renovated  6.843e+02  3.520e+02   1.944   0.0519 .
## sqft_lot      8.584e-03  6.349e-02   0.135   0.8925
## sqft_above    1.491e+02  4.617e+00  32.299  < 2e-16 ***
## sqft_basement 1.690e+02  5.477e+00  30.851  < 2e-16 ***
## bedrooms     -3.690e+04  2.342e+03 -15.756  < 2e-16 ***
## bathrooms     4.352e+04  4.052e+03  10.741  < 2e-16 ***
## floors        3.816e+04  4.482e+03   8.514  < 2e-16 ***
## grade         1.185e+05  2.642e+03  44.858  < 2e-16 ***
## sqft_living15  3.427e+01  4.263e+00   8.040  9.68e-16 ***
## sqft_lot15    -4.362e-01  9.180e-02  -4.751  2.04e-06 ***
## view_1        1.031e+05  1.399e+04   7.366  1.85e-13 ***
## view_2        4.760e+04  8.614e+03   5.525  3.34e-08 ***
## view_3        1.166e+05  1.209e+04   9.651  < 2e-16 ***
## view_4        3.099e+05  1.816e+04  17.067  < 2e-16 ***
## waterfront_1  4.363e+05  2.548e+04  17.119  < 2e-16 ***
## condition_2    4.664e+04  4.949e+04   0.942   0.3461
## condition_3    5.342e+04  4.561e+04   1.171   0.2415
## condition_4    6.581e+04  4.561e+04   1.443   0.1490
## condition_5    9.726e+04  4.588e+04   2.120   0.0340 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 212400 on 15108 degrees of freedom
## Multiple R-squared:  0.6556, Adjusted R-squared:  0.6551
## F-statistic: 1438 on 20 and 15108 DF,  p-value: < 2.2e-16

R_squared_linear<- summary(first_model)$r.squared
```

IV. Model Performance Testing

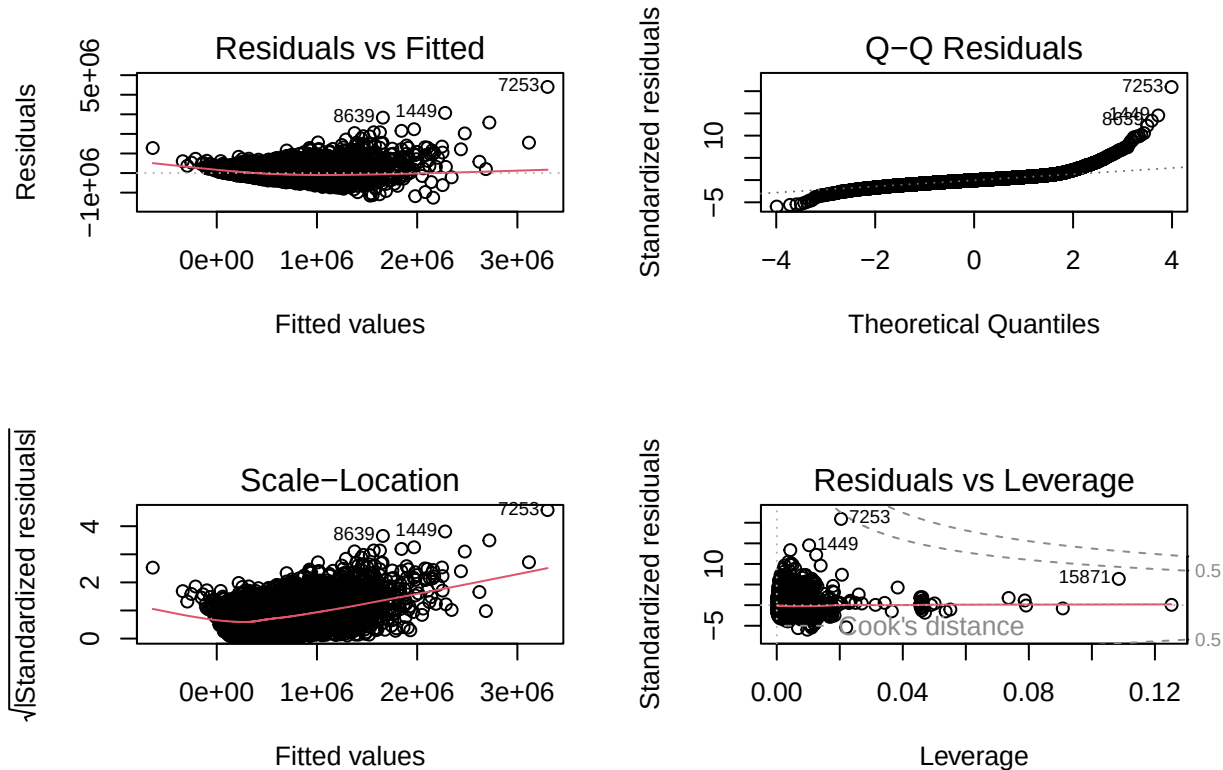
Check assumption

Before we proceed to finalize the multiple linear regression model, we conduct model diagnostic analysis visually and statistically through tests. We check the following assumptions of ordinary linear regression model as well as potential multicollinearity among the predictors.

1. Independence of the observations
2. Normal distribution of errors (residuals)
3. Homoscedasticity of the errors (residuals)
4. Linear relation between response variable and predictors
5. Independence of the observations

Since the house sold data was collected from a relatively large geographic area (King County, Washington), we assume that one property's transaction had no latent dependence on another property's transaction. For the assumption of independence of observations, we conclude that it is satisfied.

```
# Plot model summary
par(mfrow = c(2,2))
plot(first_model)
```



2. Normality of the residuals (errors) distribution

From the Q-Q plot, there is some deviation of the residuals from the theoretical quantile of normal distribution bending at the two ends of the theoretical line. We look into constructing two formal tests on the normality of residuals, the Shapiro-Wilk test and the Kalmogorov-Smirnov test. However, we find that the Shapiro-Wilk test in **stats** package only works for sample sizes less than 5000. We conduct Kalmogorov-Smirnov test.

H_0 : The residuals of model **hsreg_stepwise** follows a normal distribution

H_a : The residuals of model **hsreg_stepwise** is not normally distributed

Setting $\alpha = 0.05$, p-value < 0.05 . We reject the null hypothesis and conclude that the residuals of model **hsreg_stepwise** is not normally distributed.

```
# shapiro.test(hsreg_stepwise$residuals)
ks.test(first_model$residuals, 'pnorm')
```

```
## Warning in ks.test.default(first_model$residuals, "pnorm"): ties should not be
## present for the Kolmogorov-Smirnov test
```

```
##
## Asymptotic one-sample Kolmogorov-Smirnov test
##
## data: first_model$residuals
## D = 0.52812, p-value < 2.2e-16
## alternative hypothesis: two-sided
```

3. Homogeneity of variance (Homoscedasticity)

Taking a look at the scatter plot of residuals and fitted value of the model `hstest_reg`, there is an obvious “fan-out” shape of the residuals across different level of the linear combination of predictors (fitted value of the model). We conduct Breusch-Pagan test for confirmation of our observation on the fan-out pattern of residual variance.

H_0 : The residuals of model `hsreg_stepwise` have constant variance across levels of predictors

H_a : The residuals of model `hsreg_stepwise` have non-constant variance across levels of predictors

Setting $\alpha = 0.05$, p-value < 0.05 from Breusch-Pagan test. We reject the null hypothesis and conclude that the variance of residuals of model `hsreg_stepwise` is not constant.

```
bptest(first_model)
```

```
##
## studentized Breusch-Pagan test
##
## data: first_model
## BP = 1960.6, df = 20, p-value < 2.2e-16
```

4. Linear relation between response variable and predictors

Visually checking the Scale-Location plot, the non-parametric line between residuals and fitted values does not look straight. Thus, the data with high influential value might have pulled the relation between response variable and linear combination of predictors off from linear relation.

Variance inflation factor

Other than the four assumptions of ordinary linear regression, we also check multicollinearity in the linear model we select. We compute the variance inflation factor to quantify how much the variance of the estimated coefficient being inflated due to multicollinearity. The VIF of most predictors are less than 5, which does not signalize multicollinearity, but the age sold, age renovated, and condition 3,4,5 all have high VIF numbers.

```
vif<-vif(first_model)
d2<-data.frame(vif)
print("vif")
```

```
## [1] "vif"
```

```
d2
```

```
##              vif
## age_sold      33.852542
## age_renovated 34.574029
## sqft_lot       2.058952
## sqft_above     4.847017
## sqft_basement  1.977864
## bedrooms      1.623639
## bathrooms      3.263798
## floors         1.960734
## grade          3.217576
## sqft_living15  2.887757
```

```
## sqft_lot15      2.078150
## view_1          1.025345
## view_2          1.065089
## view_3          1.085322
## view_4          1.534757
## waterfront_1    1.472580
## condition_2     6.250179
## condition_3    159.221150
## condition_4    135.201217
## condition_5     52.214737
```

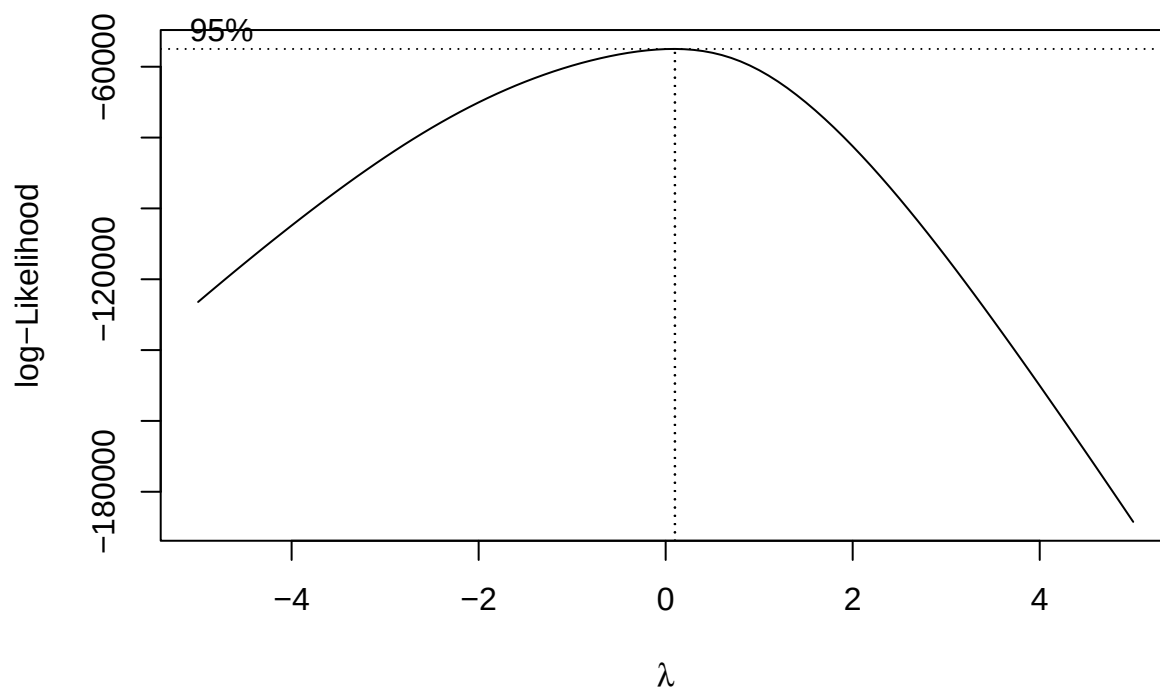
Use Box-Cox transformation to transform the distribution of house price

One-parameter Box-Cox transformation is defined as the following:

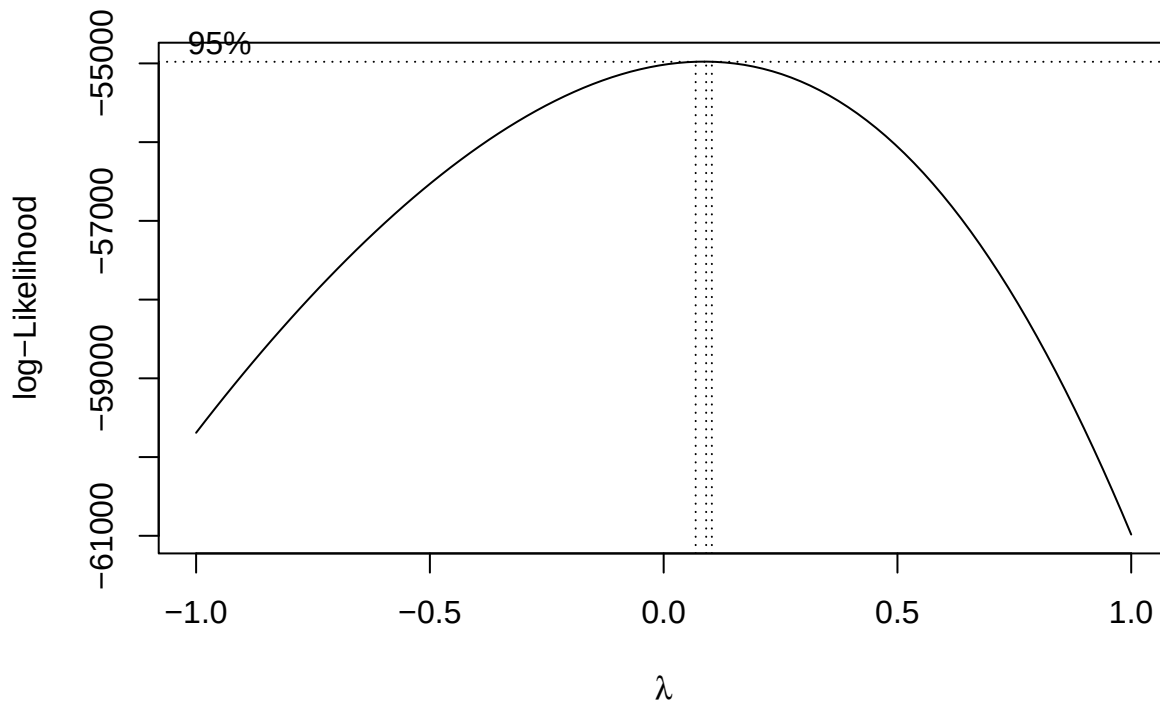
$$y_i^{(\lambda)} = \begin{cases} \frac{y_i^\lambda - 1}{\lambda} & \text{if } \lambda \neq 0 \\ \ln y_i & \text{if } \lambda = 0 \end{cases}$$

The Box-Cox transformation shows that, when the log-likelihood reaches the maximum, the log transformation might be a good transformation on the response variable.

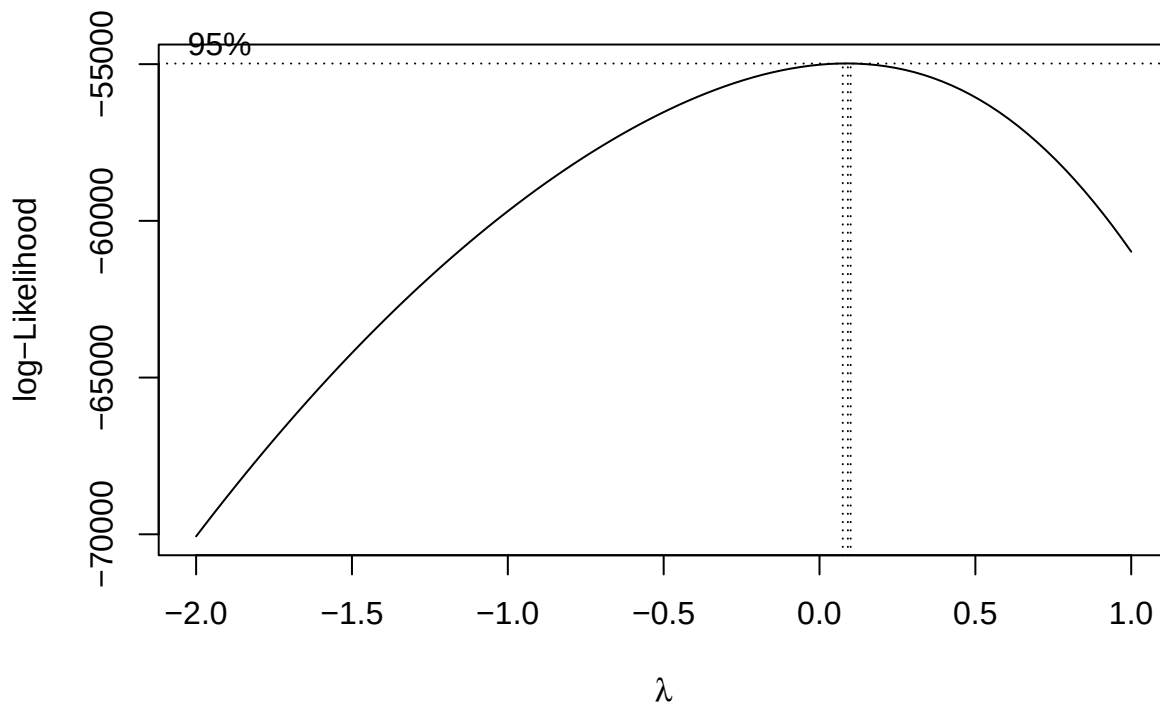
```
boxcox(first_model,lambda=seq(-5,5,by=.1))
```



```
boxcox(first_model,lambda=seq(-1,1,by=.1))
```



```
boxcox(first_model,lambda=seq(-2,1,by=.05))
```



```
f1<-lm(log(houseprice)~.,data=hstrain_noscale)
summary(f1)
```

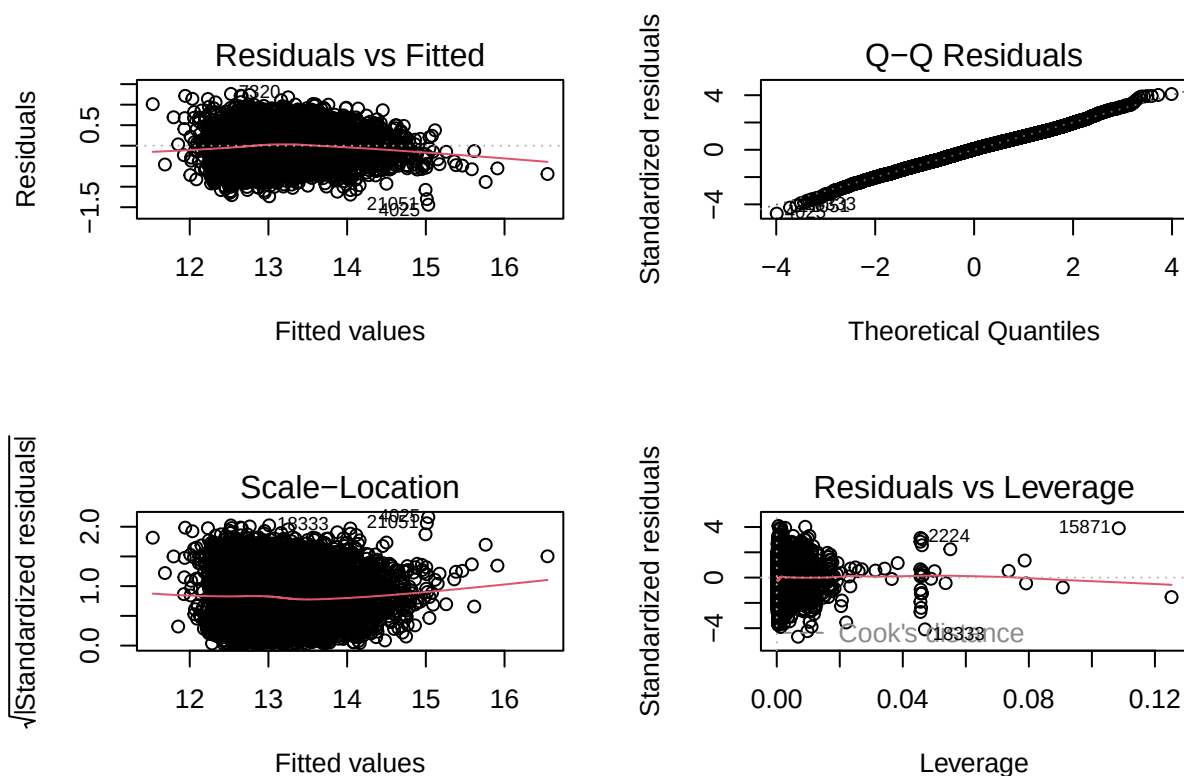
```
##
## Call:
## lm(formula = log(houseprice) ~ ., data = hstrain_noscale)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.43929 -0.20992  0.01207  0.20886  1.26145
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  1.024e+01  7.042e-02 145.443 < 2e-16 ***
## age_sold     4.330e-03  4.966e-04   8.719 < 2e-16 ***
## age_renovated 1.256e-03  5.119e-04   2.454 0.01413 *
## sqft_lot     2.819e-07  9.232e-08   3.054 0.00226 **
## sqft_above   9.851e-05  6.713e-06 14.674 < 2e-16 ***
## sqft_basement 1.944e-04  7.963e-06 24.409 < 2e-16 ***
## bedrooms    -2.312e-02  3.405e-03 -6.792 1.15e-11 ***
## bathrooms    7.512e-02  5.891e-03 12.751 < 2e-16 ***
## floors       1.284e-01  6.516e-03 19.708 < 2e-16 ***
## grade        2.010e-01  3.841e-03 52.337 < 2e-16 ***
## sqft_living15 1.180e-04  6.198e-06 19.035 < 2e-16 ***
## sqft_lot15   -3.712e-07  1.335e-07 -2.781 0.00543 **
## view_1       1.433e-01  2.035e-02   7.043 1.96e-12 ***
## view_2       7.649e-02  1.252e-02   6.107 1.04e-09 ***
## view_3       8.778e-02  1.757e-02   4.995 5.94e-07 ***
## view_4       1.984e-01  2.640e-02   7.513 6.08e-14 ***
## waterfront_1 3.119e-01  3.705e-02   8.418 < 2e-16 ***
## condition_2  4.794e-02  7.196e-02   0.666 0.50527
## condition_3  2.619e-01  6.631e-02   3.950 7.85e-05 ***
## condition_4  2.840e-01  6.631e-02   4.283 1.85e-05 ***
## condition_5  3.398e-01  6.671e-02   5.094 3.55e-07 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.3088 on 15108 degrees of freedom
## Multiple R-squared:  0.6536, Adjusted R-squared:  0.6531
## F-statistic: 1425 on 20 and 15108 DF, p-value: < 2.2e-16

f1_stepwise <- step(f1, direction = 'both')

## Start: AIC=-35530.82
## log(houseprice) ~ age_sold + age_renovated + sqft_lot + sqft_above +
##      sqft_basement + bedrooms + bathrooms + floors + grade + sqft_living15 +
##      sqft_lot15 + view_1 + view_2 + view_3 + view_4 + waterfront_1 +
##      condition_2 + condition_3 + condition_4 + condition_5
##
##              Df Sum of Sq    RSS    AIC
## - condition_2    1     0.042 1441.0 -35532
## <none>              1441.0 -35531
## - age_renovated    1     0.574 1441.5 -35527
## - sqft_lot15        1     0.737 1441.7 -35525
## - sqft_lot          1     0.889 1441.8 -35523
## - condition_3       1     1.488 1442.5 -35517
## - condition_4       1     1.750 1442.7 -35514
## - view_3            1     2.380 1443.3 -35508
## - condition_5       1     2.475 1443.4 -35507
## - view_2            1     3.557 1444.5 -35496
## - bedrooms          1     4.400 1445.4 -35487
## - view_1            1     4.731 1445.7 -35483
## - view_4            1     5.384 1446.3 -35476
```

```
## - waterfront_1 1 6.758 1447.7 -35462
## - age_sold 1 7.250 1448.2 -35457
## - bathrooms 1 15.507 1456.5 -35371
## - sqft_above 1 20.537 1461.5 -35319
## - sqft_living15 1 34.556 1475.5 -35174
## - floors 1 37.044 1478.0 -35149
## - sqft_basement 1 56.825 1497.8 -34948
## - grade 1 261.250 1702.2 -33012
##
## Step: AIC=-35532.38
## log(houseprice) ~ age_sold + age_renovated + sqft_lot + sqft_above +
## sqft_basement + bedrooms + bathrooms + floors + grade + sqft_living15 +
## sqft_lot15 + view_1 + view_2 + view_3 + view_4 + waterfront_1 +
## condition_3 + condition_4 + condition_5
##
##          Df Sum of Sq    RSS    AIC
## <none>          1441.0 -35532
## + condition_2 1 0.042 1441.0 -35531
## - age_renovated 1 0.572 1441.6 -35528
## - sqft_lot15 1 0.733 1441.7 -35527
## - sqft_lot 1 0.886 1441.9 -35525
## - view_3 1 2.382 1443.4 -35509
## - view_2 1 3.561 1444.6 -35497
## - bedrooms 1 4.387 1445.4 -35488
## - view_1 1 4.724 1445.7 -35485
## - view_4 1 5.366 1446.4 -35478
## - condition_3 1 6.519 1447.5 -35466
## - waterfront_1 1 6.757 1447.8 -35464
## - age_sold 1 7.254 1448.3 -35458
## - condition_4 1 7.826 1448.8 -35452
## - condition_5 1 10.954 1452.0 -35420
## - bathrooms 1 15.503 1456.5 -35372
## - sqft_above 1 20.538 1461.5 -35320
## - sqft_living15 1 34.514 1475.5 -35176
## - floors 1 37.013 1478.0 -35151
## - sqft_basement 1 56.832 1497.8 -34949
## - grade 1 261.745 1702.8 -33009
```

```
par(mfrow = c(2,2))
plot(f1)
```



```
box_cox<- summary(f1)$r.squared
```

To identify and remove influential observations, we check Cook's Distance and find eight observations as outliers. We delete those rows from train data frame.

```
# Assuming 'your_linear_model' is your linear regression model
d1 <- cooks.distance(first_model)
threshold <- 0.04
# Identify influential observations
influential_obs <- hstrain_noscale[d1 > threshold, ]

influential_obs
```

```
##      houseprice age_sold age_renovated sqft_lot sqft_above sqft_basement
## 8093    4668000      31          26    13068      4820          4820
## 15871    640000      67          67     6000      1040           580
## 1316   5300000      24          24    24829      5000         2390
## 2627   4500000      10          10    40014      6350          290
## 1449   5350000       6           6    23985      6720         1280
## 7253   7700000     104          77    27600      8570         3480
##      bedrooms bathrooms floors grade sqft_living15 sqft_lot15 view_1 view_2
## 8093         5       6.75    1.0   12         3270     10454      0      0
## 15871        33       1.75    1.0    7         1330      4700      0      0
## 1316         6       6.00    2.0   12         4320     24619      0      0
## 2627         5       5.50    2.0   12         3030     23408      0      0
## 1449         5       5.00    2.0   12         4600     21750      0      0
## 7253         6       8.00    2.5   13         3940      8800      0      0
##      view_3 view_4 waterfront_1 condition_2 condition_3 condition_4
## 8093        0      1           1           0           1           0
## 15871        0      0           0           0           0           0
## 1316        0      1           1           0           0           1
## 2627        0      1           1           0           1           0
## 1449        0      1           0           0           1           0
```

```
## 7253      1      0      0      0      0      1
##      condition_5
## 8093      0
## 15871     1
## 1316      0
## 2627      0
## 1449      0
## 7253      0

# Remove influential observations
# Assuming 'your_data' is your dataframe
rows_to_remove <- c(8093,15781,1316,2627,1449,6692,8639,7253) # Replace with the actual row indices you want

hstrain_noscale<- hstrain_noscale[-rows_to_remove, ]
```

Linear regression second model

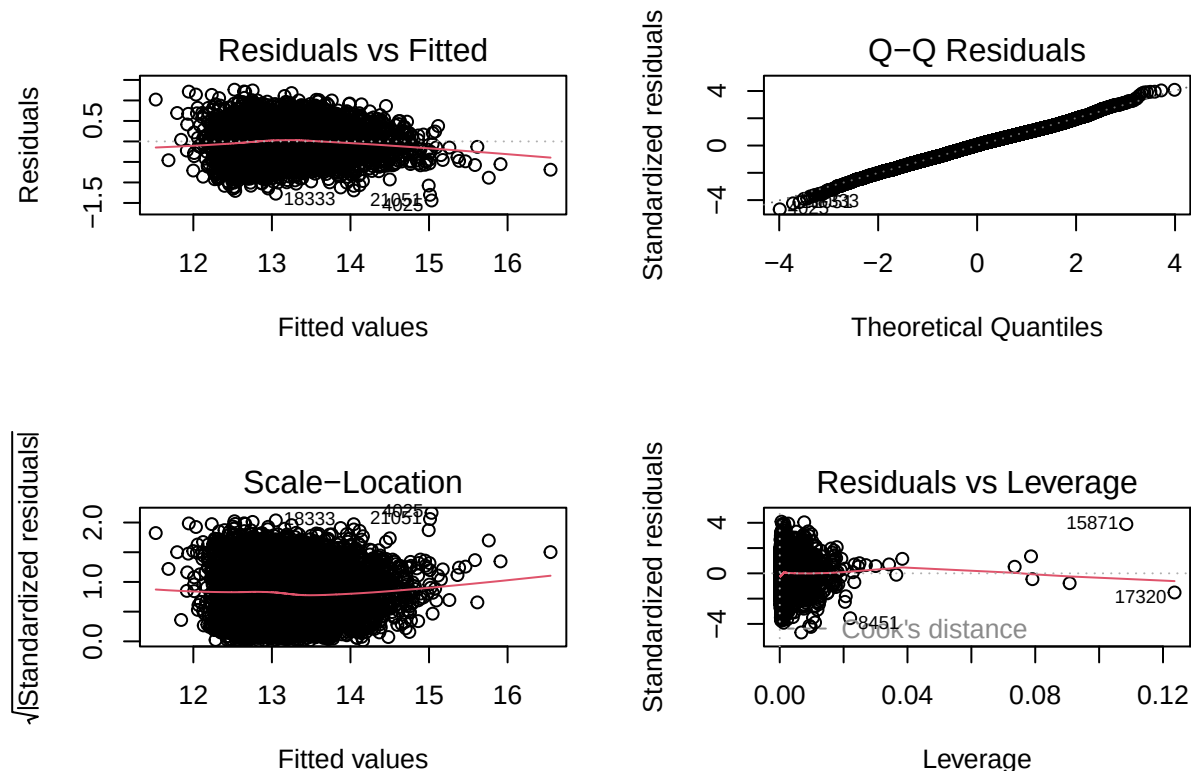
```
#remove variable condition_2 as it was not significant
second_model<-lm(log(houseprice)~age_sold+age_renovated +sqft_lot+sqft_above+sqft_basement +bedrooms
  +bathrooms+floors+grade+sqft_living15+sqft_lot15+view_1
  +view_2+view_3+view_4+waterfront_1+condition_3+condition_4 +condition_5, data=hstrain_noscale)

summary(second_model)
```

```
##
## Call:
## lm(formula = log(houseprice) ~ age_sold + age_renovated + sqft_lot +
##      sqft_above + sqft_basement + bedrooms + bathrooms + floors +
##      grade + sqft_living15 + sqft_lot15 + view_1 + view_2 + view_3 +
##      view_4 + waterfront_1 + condition_3 + condition_4 + condition_5,
##      data = hstrain_noscale)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.43979 -0.20962  0.01202  0.20902  1.26168
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  1.028e+01  3.648e-02 281.840 < 2e-16 ***
## age_sold     4.332e-03  4.966e-04   8.725 < 2e-16 ***
## age_renovated 1.254e-03  5.118e-04   2.450  0.01428 *
## sqft_lot     2.817e-07  9.231e-08   3.052  0.00228 **
## sqft_above   9.831e-05  6.714e-06  14.642 < 2e-16 ***
## sqft_basement 1.944e-04  7.964e-06  24.410 < 2e-16 ***
## bedrooms    -2.304e-02  3.405e-03  -6.768 1.36e-11 ***
## bathrooms    7.511e-02  5.892e-03  12.749 < 2e-16 ***
## floors       1.283e-01  6.517e-03  19.690 < 2e-16 ***
## grade        2.013e-01  3.840e-03  52.421 < 2e-16 ***
## sqft_living15 1.177e-04  6.195e-06  19.001 < 2e-16 ***
## sqft_lot15   -3.691e-07  1.335e-07  -2.766  0.00569 **
## view_1       1.432e-01  2.035e-02   7.036 2.07e-12 ***
## view_2       7.647e-02  1.252e-02   6.106 1.05e-09 ***
## view_3       8.773e-02  1.757e-02   4.993 6.03e-07 ***
## view_4       1.979e-01  2.640e-02   7.496 6.92e-14 ***
## waterfront_1 3.119e-01  3.705e-02   8.418 < 2e-16 ***
## condition_3  2.215e-01  2.680e-02   8.265 < 2e-16 ***
## condition_4  2.435e-01  2.690e-02   9.053 < 2e-16 ***
## condition_5  2.993e-01  2.794e-02  10.712 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
##
## Residual standard error: 0.3088 on 15102 degrees of freedom
## Multiple R-squared:  0.6537, Adjusted R-squared:  0.6532
## F-statistic: 1500 on 19 and 15102 DF,  p-value: < 2.2e-16
```

```
par(mfrow = c(2,2))
plot(second_model)
```



```
bptest(second_model)
```

```
##
## studentized Breusch-Pagan test
##
## data: second_model
## BP = 275.25, df = 19, p-value < 2.2e-16
```

Other than transformations, we apply other remedial measures, such as Ridge, ElasticNet, and Lasso, weighted least square regression (WLS), Huber robust regression, Robust Linear Regression.

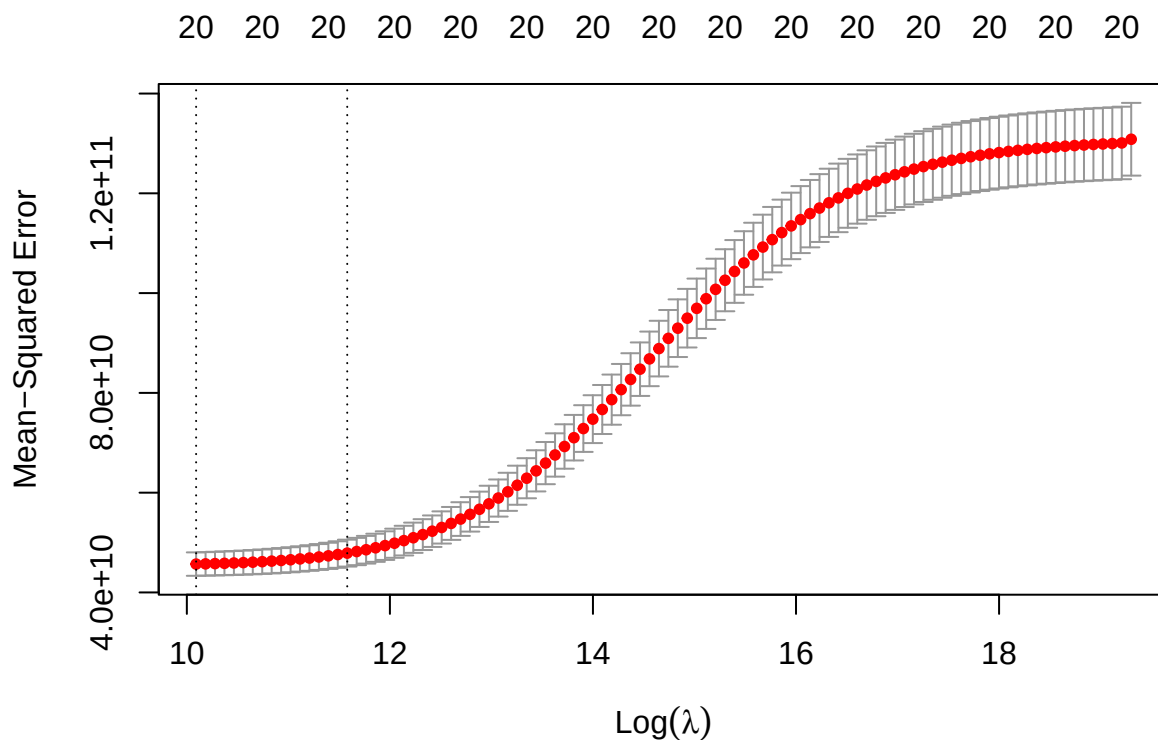
Ridge Regression Model for House Price Prediction

We build a ridge regression model to predict house prices based on various features. Ridge regression is chosen for its ability to handle multicollinearity and prevent over fitting. We start by preparing the data for modeling. The model matrix (x) is created by excluding the intercept term to avoid collinearity issues. The response variable (y) is the house prices. We perform cross-validation to find the optimal lambda for our ridge regression model. The alpha parameter is set to 0 for ridge regression, and we test a range of lambda values. A plot is generated to visualize the mean cross-validated error across different lambda values. This helps us identify the lambda that minimizes the error. We extract the coefficients for the chosen lambda, thus representing the impact of each predictor on the house prices. The final step involves printing the coefficients which provides insights into the importance and direction of each feature in the model.

```
x<-model.matrix(houseprice~., hstrain_noscale)[,-c(1)]
y <- hstrain_noscale$houseprice

# Cross Validate and calculate best lambda
```

```
Ridge_regression <- cv.glmnet(x, y, alpha=0, nlambda=100, lambda.min.ratio=0.0001)
best_lambda <- Ridge_regression$lambda.min
plot(Ridge_regression)
```



```
coeff_ridge_regression <- coef(Ridge_regression , s = best_lambda)
print (coeff_ridge_regression)
```

```
## 21 x 1 sparse Matrix of class "dgCMatrix"
##              s1
## (Intercept) -8.934977e+05
## age_sold    1.939159e+03
## age_renovated 1.497481e+03
## sqft_lot     2.343627e-02
## sqft_above   1.256662e+02
## sqft_basement 1.480558e+02
## bedrooms    -2.741516e+04
## bathrooms    5.012872e+04
## floors       4.017316e+04
## grade        1.084266e+05
## sqft_living15 5.234488e+01
## sqft_lot15   -3.572368e-01
## view_1       1.040135e+05
## view_2       5.342308e+04
## view_3       1.214950e+05
## view_4       3.150554e+05
## waterfront_1 4.287098e+05
## condition_2  -1.921514e+04
## condition_3  -8.685785e+03
## condition_4   4.627210e+03
## condition_5   3.662924e+04
```

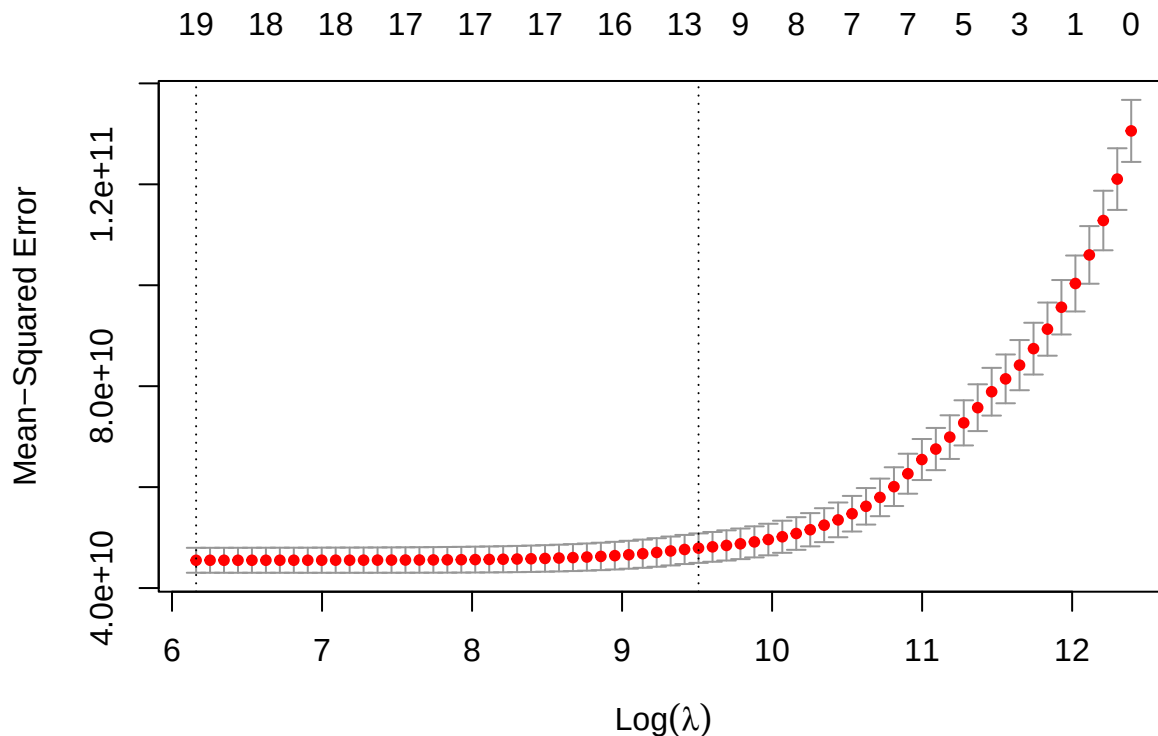
```
# Predict the response variable using the Ridge regression model
y_pred <- predict(Ridge_regression, newx = x, s = best_lambda)

mse_model <- mean((y - y_pred)^2)
null_model <- lm(y ~ 1)
y_pred_null <- predict(null_model)
mse_null <- mean((y - y_pred_null)^2)
R_squared_ride <- 1 - mse_model / mse_null
```

Lasso Regression Model for House Price Prediction

This code implements a Lasso Regression model for predicting house prices based on a set of features. Lasso regression is selected for its ability to perform variable selection by encouraging some coefficients to be exactly zero. This can be beneficial when dealing with data sets with a large number of potential predictors. The variables 'x' and 'y' represent the model matrix (predictors) and the response variable (house prices), respectively. We use cross-validation to find the optimal lambda for our Lasso Regression model. The alpha parameter is set to 1 for Lasso regression. A range of lambda values is tested, and lambda.min.ratio controls the range of lambda values. Then we create a plot for visualization of the mean cross-validated error across different lambda values. This aids in identifying the lambda that minimizes the error and provides a sparse model. We extract the coefficients for the chosen lambda, representing the impact of each predictor on house prices. The final step involves printing the coefficients for interpretation and analysis.

```
lasso_regression <- cv.glmnet(x, y, alpha = 1, nlambda=100, lambda.min.ratio=0.0001)
best_lambda.lasso <- lasso_regression$lambda.min
plot(lasso_regression)
```



```
coeff_lasso_regression <- coef(lasso_regression, s = best_lambda.lasso)
print(coeff_lasso_regression)
```

```
## 21 x 1 sparse Matrix of class "dgCMatrix"
##           s1
## (Intercept) -9.413524e+05
## age_sold    3.059356e+03
```

```
## age_renovated 5.714532e+02
## sqft_lot .
## sqft_above 1.476521e+02
## sqft_basement 1.675884e+02
## bedrooms -3.523550e+04
## bathrooms 4.276575e+04
## floors 3.655932e+04
## grade 1.191393e+05
## sqft_living15 3.380866e+01
## sqft_lot15 -4.035773e-01
## view_1 9.975188e+04
## view_2 4.597607e+04
## view_3 1.144523e+05
## view_4 3.077772e+05
## waterfront_1 4.331035e+05
## condition_2 -1.233151e+04
## condition_3 -1.053610e+04
## condition_4 6.218103e+02
## condition_5 3.191194e+04

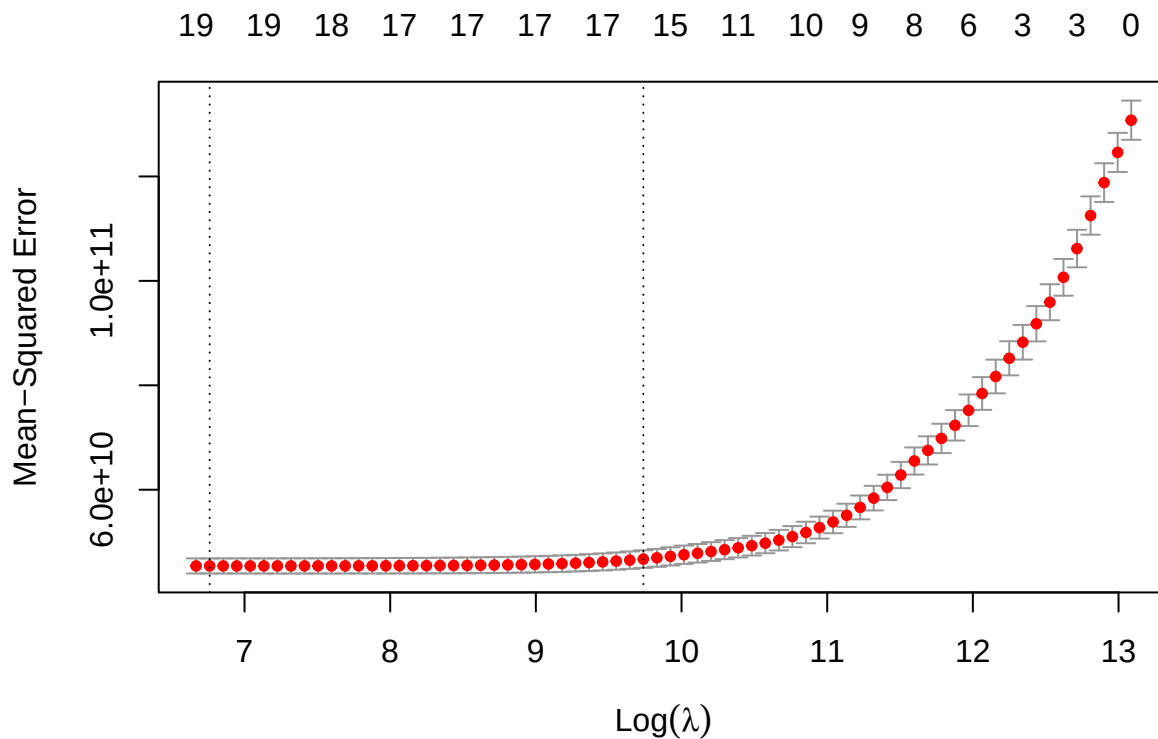
# Predict the response variable using the Ridge regression model
y_pred <- predict(lasso_regression, newx = x, s = best.lambda.lasso)

mse_model <- mean((y - y_pred)^2)
null_model <- lm(y ~ 1)
y_pred_null <- predict(null_model)
mse_null <- mean((y - y_pred_null)^2)
R_squared_lasso <- 1 - mse_model / mse_null
```

Elastic Net Regression Model for House Price Prediction

This code implements an Elastic Net Regression model for predicting house prices based on a set of features. Elastic Net combines Lasso and Ridge regularization techniques, offering a balance between feature selection and handling multicollinearity. The variables 'x' and 'y' are same as ridge and lasso regression. We use cross-validation to find the optimal lambda for our Elastic Net Regression model. The alpha parameter is set to 0.5, representing a mixture of Lasso and Ridge regularization. A range of lambda values is tested, and lambda.min.ratio controls the range of lambda values and then plot generates a visualization of the mean cross-validated error across different lambda values. This helps identify the lambda that minimizes the error and provides insights into the model's performance. We extract the coefficients for the chosen lambda, representing the impact of each predictor on house prices. Elastic Net, with its combined Lasso and Ridge penalties, allows for both variable selection and handling multicollinearity. The final step involves printing the coefficients for interpretation and analysis.

```
elastic_net <- cv.glmnet(x, y, alpha = 0.5, nlambda=100, lambda.min.ratio=0.0001)
lambda.elastic_net <- elastic_net$lambda.min
plot(elastic_net)
```



```
coeff_elastic_net <- coef(elastic_net , s = lambda.elastic_net)
print (coeff_elastic_net)
```

```
## 21 x 1 sparse Matrix of class "dgCMatrix"
##              s1
## (Intercept)  -9.407214e+05
## age_sold      2.861304e+03
## age_renovated  7.725030e+02
## sqft_lot       .
## sqft_above     1.472318e+02
## sqft_basement  1.672585e+02
## bedrooms      -3.520329e+04
## bathrooms      4.307902e+04
## floors         3.684470e+04
## grade          1.188602e+05
## sqft_living15  3.427085e+01
## sqft_lot15     -4.029792e-01
## view_1         1.001342e+05
## view_2         4.636397e+04
## view_3         1.148448e+05
## view_4         3.082568e+05
## waterfront_1   4.341531e+05
## condition_2    -1.270650e+04
## condition_3    -1.033725e+04
## condition_4     8.965447e+02
## condition_5     3.214668e+04
```

```
# Predict the response variable using the Ridge regression model
y_pred <- predict(elastic_net, newx = x, s = lambda.elastic_net)
```

```
mse_model <- mean((y - y_pred)^2)
null_model <- lm(y ~ 1)
y_pred_null <- predict(null_model)
```

```
mse_null <- mean((y - y_pred_null)^2)
R_squared_elasticnet <- 1 - mse_model / mse_null
```

Weighted Linear Regression Model for House Price Prediction

This code implements a weighted linear regression model for predicting house prices. Weighted regression assigns different weights to individual observations, allowing for modeling heteroscedasticity or addressing data points with varying precision. Initially, a standard linear regression model (f2) is fitted to the training data (Train_data) without considering weights. The fitted values (si) and weights (wi) are then calculated based on the residuals of the initial model. After that a new linear regression model (weighted_reg) is fitted, incorporating the calculated weights (wi) to account for varying precision among observations. The weights are inversely proportional to the squared fitted values, emphasizing data points with larger residuals. Observations with higher weights contribute more to the estimation of coefficients and have a greater impact on the overall fit of the model. Observations with lower weights have less influence on the model.

```
f2<-lm(log(houseprice)~.,hstrain_noscale)
si<-f2$fitted.values
wi<-1/si^2

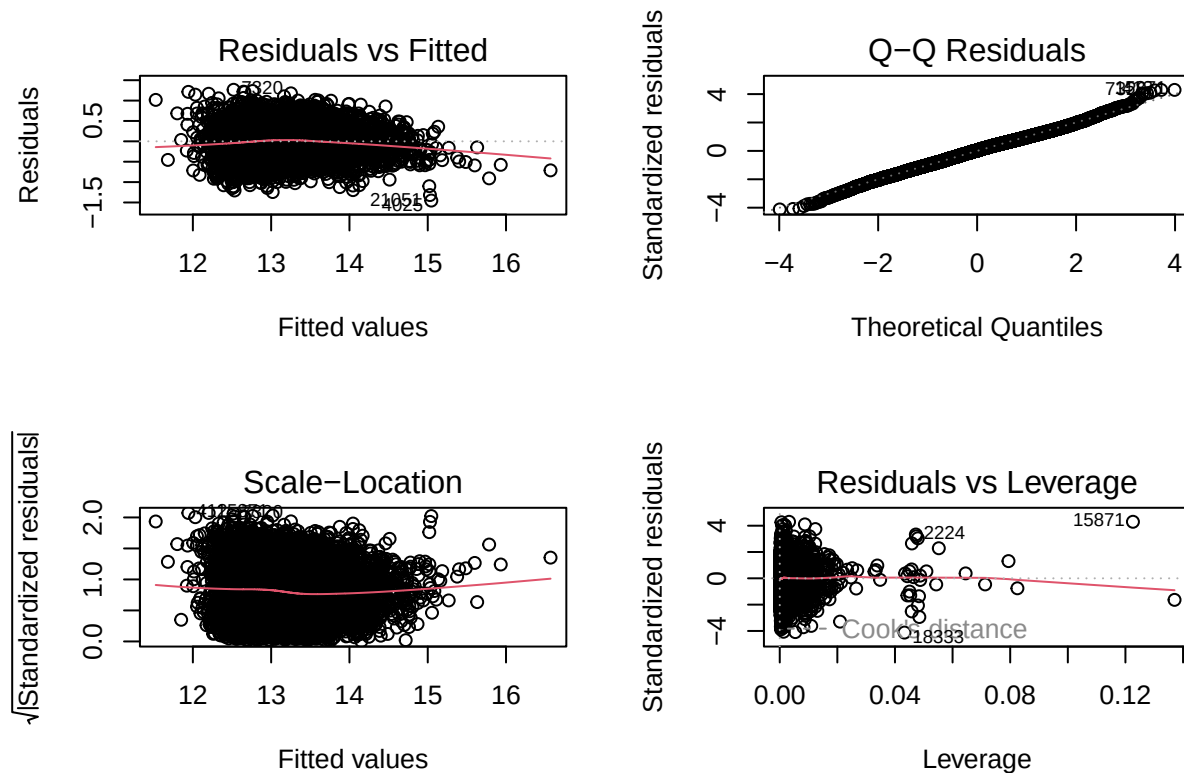
weighted_reg<-lm(log(houseprice)~.,data=hstrain_noscale, weights=wi)
summary(weighted_reg)
```

```
##
## Call:
## lm(formula = log(houseprice) ~ ., data = hstrain_noscale, weights = wi)
##
## Weighted Residuals:
##      Min       1Q   Median       3Q      Max
## -0.096680 -0.016065  0.000841  0.015968  0.101656
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  1.023e+01  6.781e-02 150.899 < 2e-16 ***
## age_sold     4.294e-03  4.990e-04   8.605 < 2e-16 ***
## age_renovated 1.317e-03  5.138e-04   2.563  0.01039 *
## sqft_lot     2.906e-07  9.378e-08   3.099  0.00195 **
## sqft_above   9.893e-05  6.866e-06 14.408 < 2e-16 ***
## sqft_basement 1.986e-04  8.147e-06 24.372 < 2e-16 ***
## bedrooms    -2.400e-02  3.399e-03 -7.062 1.72e-12 ***
## bathrooms    7.486e-02  5.951e-03 12.578 < 2e-16 ***
## floors       1.304e-01  6.543e-03 19.927 < 2e-16 ***
## grade        2.017e-01  3.872e-03 52.098 < 2e-16 ***
## sqft_living15 1.207e-04  6.325e-06 19.080 < 2e-16 ***
## sqft_lot15   -3.398e-07  1.346e-07 -2.524  0.01161 *
## view_1       1.405e-01  2.097e-02  6.699 2.17e-11 ***
## view_2       7.722e-02  1.286e-02  6.005 1.95e-09 ***
## view_3       8.737e-02  1.830e-02  4.775 1.81e-06 ***
## view_4       2.004e-01  2.807e-02  7.142 9.61e-13 ***
## waterfront_1 3.097e-01  3.966e-02  7.808 6.19e-15 ***
## condition_2  4.355e-02  6.886e-02  0.632  0.52714
## condition_3  2.571e-01  6.345e-02  4.052 5.11e-05 ***
## condition_4  2.778e-01  6.345e-02  4.378 1.20e-05 ***
## condition_5  3.329e-01  6.388e-02  5.211 1.90e-07 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.02371 on 15101 degrees of freedom
## Multiple R-squared:  0.6405, Adjusted R-squared:  0.64
## F-statistic: 1345 on 20 and 15101 DF, p-value: < 2.2e-16
```

```
R_squared_weighted<-summary(weighted_reg)$r.squared
bptest(weighted_reg, studentize = FALSE)
```

```
##
## Breusch-Pagan test
##
## data: weighted_reg
## BP = 1325028, df = 20, p-value < 2.2e-16
```

```
par(mfrow = c(2,2))
plot(weighted_reg)
```



Huber Robust Regression

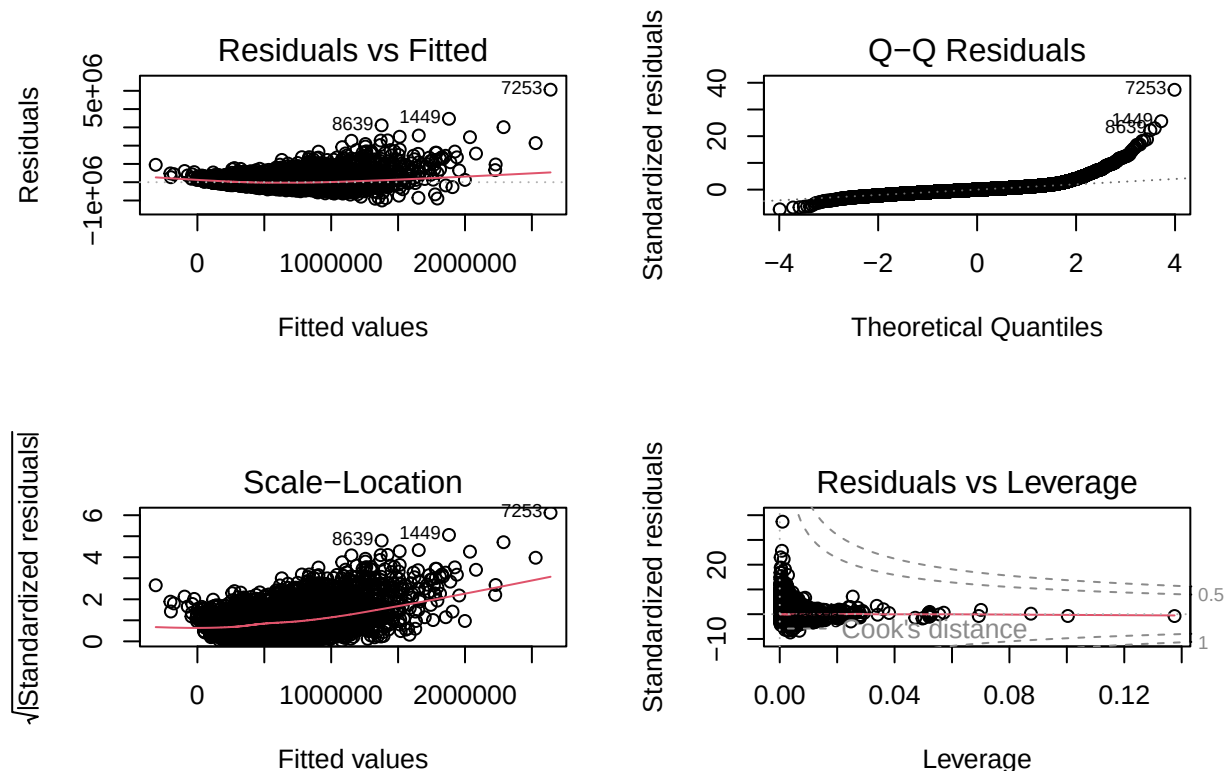
Robust Linear Regression with Huber M-Estimation This code implements a robust linear regression model using Huber M-estimation. Huber M-estimation is a robust regression technique that combines the advantages of ordinary least squares (OLS) and absolute deviation (L1) regression. It is less sensitive to the influence of outliers compared to OLS. The 'psi' argument is set to 'psi.huber', indicating the use of the Huber M-estimator for robust standard errors. Robust standard errors help produce more reliable estimates, especially when the assumptions of homoscedasticity and normality are violated. The 'summary()' function is used to display key statistics and coefficients for the Huber M-estimation model.

```
longley.reg.huber <- rlm(houseprice ~ ., psi = psi.huber, data = hstrain_noscale)
summary(longley.reg.huber)
```

```
##
## Call: rlm(formula = houseprice ~ ., data = hstrain_noscale, psi = psi.huber)
## Residuals:
##      Min       1Q   Median       3Q      Max
## -999542  -92060   -2555   90527  5060528
##
## Coefficients:
##              Value      Std. Error  t value
## (Intercept) -903596.3115    33465.1594   -27.0011
```

```
## age_sold      2108.6842      235.9971      8.9352
## age_renovated 1078.6378      243.2571      4.4341
## sqft_lot      0.0902       0.0439      2.0556
## sqft_above    88.8899      3.1908     27.8583
## sqft_basement 125.1593      3.7849     33.0684
## bedrooms     -25641.5553     1618.2111    -15.8456
## bathrooms     32712.7418     2800.0021     11.6831
## floors        55250.0311     3097.3729     17.8377
## grade         107492.6069     1826.1314     58.8636
## sqft_living15  50.3477       2.9461     17.0897
## sqft_lot15    -0.2337       0.0634     -3.6845
## view_1        74390.1739     9670.1998      7.6927
## view_2        52400.3750     5952.1218      8.8036
## view_3        106375.4619     8351.1753     12.7378
## view_4        257478.1372    12547.0592    20.5210
## waterfront_1  362646.9966     17609.0045    20.5944
## condition_2    45684.3246     34199.4868      1.3358
## condition_3    85366.4745     31514.4700      2.7088
## condition_4    94810.6370     31513.9871      3.0085
## condition_5   128375.3951     31702.4843      4.0494
##
## Residual standard error: 135500 on 15101 degrees of freedom
```

```
par(mfrow = c(2,2))
plot(longley.reg.huber)
```



```
residuals_model <- residuals(longley.reg.huber)
mse_model <- mean(residuals_model^2)

null_model <- lm(houseprice ~ 1, data = hstrain_noscale)
residuals_null <- residuals(null_model)
mse_null <- mean(residuals_null^2)
r_squared_huber <- 1 - mse_model / mse_null
```

Robust Linear Regression with Bisquare M-Estimation

This code implements a robust linear regression model using Bisquare M-estimation. The Bisquare M-estimator is a robust regression technique that minimizes the influence of outliers on parameter estimation, making it less sensitive to extreme values in the dataset. The `rlm()` function from the 'MASS' package is used to fit a linear regression model with Bisquare M-estimation. The model is specified as `'price ~ .'`, indicating that house prices are regressed on all available predictor variables in the 'Train_data' dataset. The `'psi'` argument is set to `'psi.bisquare'`, indicating the use of the Bisquare M-estimator for robust standard errors. Robust standard errors help produce more reliable estimates, especially in the presence of outliers. The `'summary()'` function is used to display key statistics and coefficients for the Bisquare M-estimation model.

```
rr.bisquare <- rlm(houseprice~., data=hstrain_noscale, psi = psi.bisquare)
summary(rr.bisquare)

##
## Call: rlm(formula = houseprice ~ ., data = hstrain_noscale, psi = psi.bisquare)
## Residuals:
##      Min       1Q   Median       3Q      Max
## -724940  -84984    2929   95820  5373528
##
## Coefficients:
##              Value      Std. Error  t value
## (Intercept) -820256.7790    32102.0078   -25.5516
## age_sold      1809.9140     226.3841     7.9949
## age_renovated  1163.2587     233.3484     4.9851
## sqft_lot        0.1218       0.0421     2.8931
## sqft_above      63.2284       3.0608    20.6574
## sqft_basement   103.5869       3.6307    28.5309
## bedrooms     -20457.7212    1552.2958   -13.1790
## bathrooms      29933.4873    2685.9484    11.1445
## floors         59204.6210    2971.2062    19.9261
## grade          99789.6733    1751.7468    56.9658
## sqft_living15    59.4206       2.8261    21.0258
## sqft_lot15      -0.1680       0.0609    -2.7611
## view_1         61395.0162    9276.2991     6.6185
## view_2         48435.7836    5709.6713     8.4831
## view_3         83756.5305    8011.0030    10.4552
## view_4        215392.8905   12035.9741    17.8958
## waterfront_1   151182.6174   16891.7289     8.9501
## condition_2     23653.6559   32806.4235     0.7210
## condition_3     76714.8793   30230.7768     2.5376
## condition_4     84822.0635   30230.3136     2.8059
## condition_5    117172.1975   30411.1326     3.8529
##
## Residual standard error: 133400 on 15101 degrees of freedom
```

```
#calculating R square for model
residuals_model <- residuals(rr.bisquare)
mse_model <- mean(residuals_model^2)
null_model <- lm(houseprice ~ 1, data = hstrain_noscale)
residuals_null <- residuals(null_model)
mse_null <- mean(residuals_null^2)

R_squared_bisquare <- round((1 - mse_model / mse_null),4)
```

Model performance

By looking at the results, the best model is the linear regression model with the highest R square value of 0.655, and the lasso and box cox transformation has approximately same results with R square 0.6528333.

```
r_squared_df1 <- data.frame(
  Model = c("Linear Regression", "LASSO", "Ridge Regression", "Elastic Net", "Huber Weights", "Bisquare Weight",
  R_Squared = c(R_squared_linear, R_squared_lasso, R_squared_ridge, R_squared_elasticnet, r_squared_huber, R_squared_bisquare)
```

```
)

print("R squared of all models created")

## [1] "R squared of all models created"
r_squared_df1
```

```
##           Model R_Squared
## 1  Linear Regression 0.6555660
## 2           LASSO 0.6555356
## 3  Ridge Regression 0.6533199
## 4    Elastic Net 0.6555343
## 5   Huber Weights 0.6323327
## 6   Bisquare Weight 0.5959000
## 7 Weighted Regression 0.6405230
## 8    BoxCox model 0.6535776
```

Model Performance and out of sample testing

After evaluating the performance of various models using test data set, we determine that LASSO yielded the highest effectiveness in predicting house prices, with an R-squared value of approximately 0.6559796. This value indicates that around 65.60% of the variability in house prices is explained by the LASSO, signifying a favorable fit to the data.

```
library(dplyr)
result<-data.frame()
predictors <- data.matrix(dplyr::select(hstest_noscale, -houseprice))
y<-hstest_noscale$houseprice

y1<-predict(first_model, data.frame(hstest_noscale) )

y2<-predict(lasso_regression, s = best.lambda.lasso, newx= predictors)

y3<-predict(Ridge_regression, s = best_lambda, newx=predictors )

y4<-predict(elastic_net, s = lambda.elastic_net, newx= predictors)

#y5<-predict(step_both.final, data.frame(hstest_noscale))

y6<-predict(longley.reg.huber, data.frame(hstest_noscale))

y7<-predict(rr.bisquare, data.frame(hstest_noscale))

# performance of model
#calculate sst
sst <- sum((y - mean(y))^2)

# Linear Regression
sse.lm <- sum((y - y1)^2)
R.Square.lm <- 1 - (sse.lm / sst)

# LASSO
sse.lasso <- sum((y - y2)^2)
R.Square.lasso <- 1 - (sse.lasso / sst)

# Ridge Regression
sse.ridge <- sum((y - y3)^2)
R.Square.ridge <- 1 - (sse.ridge / sst)
```

```

# Elastic Net
sse.enet <- sum((y - y4)^2)
R.Square.Enet <- 1 - (sse.enet / sst)

# stepwise
#sse.stepwise <- sum((y - y5)^2)
#R.Square.Step<- 1 - (sse.enet / sst)

# Huber Weights
sse.huber <- sum((y - y6)^2)
R.Square.huber <- 1 - (sse.huber / sst)

#bisquare weights
sse.bisquare <- sum((y - y7)^2)
R.Square.bisquare <- 1 - (sse.bisquare / sst)

# Create a dataframe
r_squared_df <- data.frame(
  Model = c("Linear Regression", "LASSO", "Ridge Regression", "Elastic Net", "Huber Weights", "Bisquare Weight"),
  R_Squared = c(R.Square.lm, R.Square.lasso, R.Square.ridge, R.Square.Enet, R.Square.huber, R.Square.bisquare)
)

print("R squared of test dataset")

## [1] "R squared of test dataset"
r_squared_df

##           Model R_Squared
## 1 Linear Regression 0.6559345
## 2           LASSO 0.6559796
## 3 Ridge Regression 0.6511818
## 4 Elastic Net 0.6559668
## 5 Huber Weights 0.6229190
## 6 Bisquare Weight 0.5782598

```

V. Challenger Models (15 points)

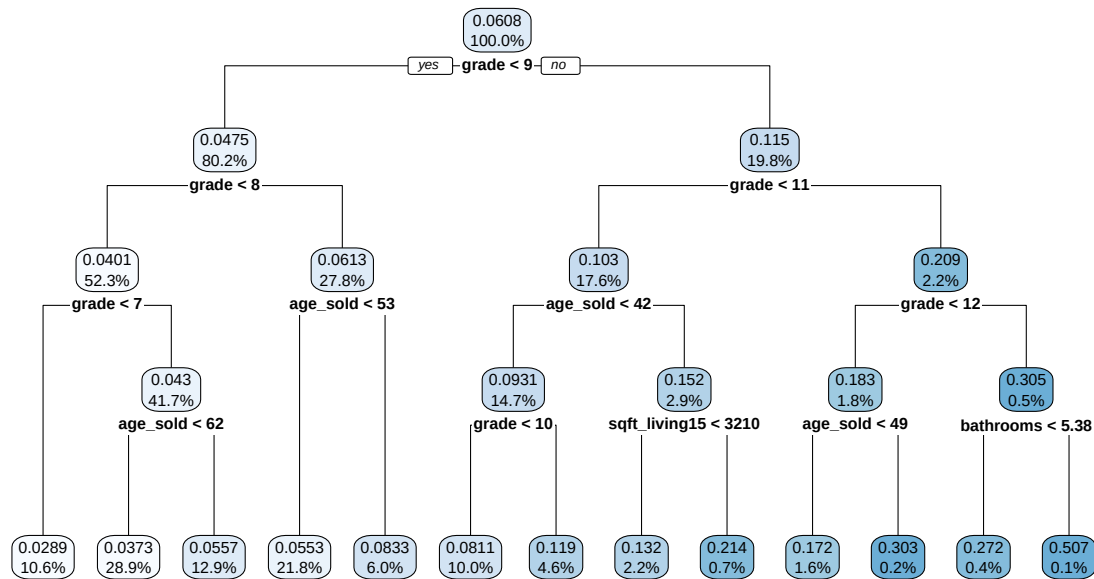
The regression tree model is based on the train dataset and then tests using train and test data similar to the process for the linear regression model. The function `rpart` is used and the resulting regression tree showed a correlation of 0.77 between predicted and actual price. The tree plot is shown below. From the tree, the grade is the most important variable in terms of predicting price. R^2 is also calculated and compared between train and test data. The train dataset R^2 is 0.61 and test data set R^2 is 0.59. The R^2 is acceptable and model is consistent with test and train samples. Compared to the linear regression, the regression tree is slightly less accurate in predicting the sale price.

```
#regression tree

m.rpart <- rpart(houseprice_normlz ~ ., data = hstrain[, -1])
m.rpart

## n= 15129
##
## node), split, n, deviance, yval
##      * denotes terminal node
##
## 1) root 15129 34.0365500 0.06081489
##    2) grade< 8.5 12131 7.9136350 0.04747836
##      4) grade< 7.5 7920 3.3590010 0.04012888
##        8) grade< 6.5 1607 0.4037842 0.02887855 *
##        9) grade>=6.5 6313 2.7000430 0.04299270
##          18) age_sold< 61.5 4367 1.2209190 0.03731529 *
##          19) age_sold>=61.5 1946 1.0224820 0.05573332 *
##      5) grade>=7.5 4211 3.3222420 0.06130116
##        10) age_sold< 52.5 3304 1.6977320 0.05525013 *
##        11) age_sold>=52.5 907 1.0628460 0.08334371 *
##    3) grade>=8.5 2998 15.2345800 0.11477940
##      6) grade< 10.5 2660 7.3799500 0.10284090
##        12) age_sold< 41.5 2219 4.0238850 0.09308191
##          24) grade< 9.5 1517 1.4126640 0.08111149 *
##          25) grade>=9.5 702 1.9241150 0.11894960 *
##        13) age_sold>=41.5 441 2.0813450 0.15194600
##          26) sqft_living15< 3210 332 0.8743658 0.13171110 *
##          27) sqft_living15>=3210 109 0.6569914 0.21357890 *
##      7) grade>=10.5 338 4.4919130 0.20873260
##        14) grade< 11.5 268 1.8956650 0.18346320
##          28) age_sold< 48.5 245 1.3607820 0.17220320 *
##          29) age_sold>=48.5 23 0.1729342 0.30340640 *
##        15) grade>=11.5 70 1.7699340 0.30547850
##          30) bathrooms< 5.375 60 0.7632563 0.27194150 *
##          31) bathrooms>=5.375 10 0.5342928 0.50670020 *

rpart.plot(m.rpart, digits = 3)
```



#evaluating model performance

```
p.rpart <- predict(m.rpart, hstest)
summary(p.rpart)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.02888 0.03732 0.05525 0.06104 0.08111 0.50670
```

```
summary(hstest$houseprice_normlz)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.0007869 0.0327869 0.0491803 0.0614158 0.0751475 0.9163934
```

```
cor(p.rpart, hstest$houseprice_normlz)
```

```
## [1] 0.7683305
```

#Measuring performance with the mean absolute error

```
MAE <- function(actual, predicted) {mean(abs(actual - predicted))}
MAE(hstest$houseprice_normlz,p.rpart)
```

```
## [1] 0.0194598
```

#Measuring performance with the SSE

```
SSE <- function(actual, predicted) {sum((actual - predicted)^2)}
SSE(hstest$houseprice_normlz,p.rpart)
```

```
## [1] 6.581859
```

#Measuring performance with the RSquare

```
R2 <- function(actual, predicted) {sum((actual - predicted)^2)/((length(actual)-1)*var(actual))}
1-R2(hstest$houseprice_normlz,p.rpart)
```

```
## [1] 0.5902483
```

#evaluating model performance for train dataset

```
p.rpart.train <- predict(m.rpart, hstrain)
summary(p.rpart.train)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.02888 0.03732 0.05525 0.06081 0.08111 0.50670
```

```
summary(hstrain$houseprice_normlz)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.00000 0.03213 0.04918 0.06081 0.07426 1.00000
```

```
cor(p.rpart.train, hstrain$houseprice_normlz)
```

```
## [1] 0.7841614
```

```
#Measuring performance with the mean absolute error
```

```
MAE <- function(actual, predicted) {mean(abs(actual - predicted))}
```

```
MAE(hstrain$houseprice_normlz,p.rpart.train)
```

```
## [1] 0.01921132
```

```
#Measuring performance with the SSE
```

```
SSE <- function(actual, predicted) {sum((actual - predicted)^2)}
```

```
SSE(hstrain$houseprice_normlz,p.rpart.train)
```

```
## [1] 13.10716
```

```
#Measuring performance with the RSquare
```

```
R2 <- function(actual, predicted) {sum((actual - predicted)^2)/((length(actual)-1)*var(actual))}
```

```
1-R2(hstrain$houseprice_normlz,p.rpart.train)
```

```
## [1] 0.6149091
```

A neural network works similarly to the human brain's neural network. A "neuron" in a neural network is a mathematical function that collects and classifies information according to a specific architecture. The network bears a strong resemblance to statistical methods such as curve fitting and regression analysis. A neural network contains layers of interconnected nodes. Each node is known as a perception and is similar to a multiple linear regression. The perception feeds the signal produced by a multiple linear regression into an activation function that may be nonlinear.

Application of Neural Networks Neural networks work best when the input data are scaled to a narrow range around zero, and here we see values ranging anywhere from zero to over a thousand. Hence let's recreate train and test dataset with normalized values:

```
hssize <- dim(hs_keyvars)[1]
```

```
set.seed(1023)
```

```
sample_indx <- sample(c(1:hssize), hssize*0.7)
```

```
# Sample according to the index
```

```
# Exclude age_renovated
```

```
hs <- hs_keyvars %>%
```

```
  dplyr::select(-c(houseprice_normlz, age_renovated)) # removed houseprice_normlz
```

```
hstrain <- hs[sample_indx,]
```

```
hstest <- hs[-sample_indx,]
```

```
normalize <- function(x) {return((x - min(x)) / (max(x) - min(x)))}
```

```
hs_norm <- as.data.frame(lapply(hs_keyvars, normalize))
```

```
hssize <- dim(hs_norm)[1]
```

```
set.seed(1023)
```

```
sample_indx <- sample(c(1:hssize), hssize*0.7)
```

```
# Sample according to the index
```

```
# Exclude age_renovated
```

```
hs <- hs_norm %>%
```

```
  dplyr::select(-c(houseprice_normlz, age_renovated))
```

```
hstrain_norm <- hs[sample_indx,]
```

```
hstest_norm <- hs[-sample_indx,]
```

```
unnormailize <- function(x) {return((x * (max(hs_keyvars$houseprice)) -min(hs_keyvars$houseprice)) + min(hs_key
```

First Model

In this simple model, there is one input node for each of the fourteen features, followed by a single hidden node and a single output node that predicts the price of the houses in king county, Washington.

```
nn_hs_model <- neuralnet(houseprice ~ ., data = hstrain_norm)
```

The network topology diagram gives us a peek into the black box of the ANN, but it doesn't provide much information about how well the model fits future data:

```
plot(nn_hs_model)
```

We'll measure the correlation between our predicted concrete strength and the true value. If the predicted and actual values are highly correlated, the model is likely to be a useful gauge of concrete strength

```
nn_model_results <- neuralnet::compute(nn_hs_model, hstrain_norm[-1])
predicted_hs <- nn_model_results$net.result
```

```
cor(predicted_hs, hstrain_norm$houseprice)
```

```
##           [,1]
## [1,] 0.8426381
```

```
pred_hs_org <- unnormalize(predicted_hs)
PredictedTest<-pred_hs_org
```

```
NN_ModelTest1<-data.frame(obs = hstrain$houseprice, pred=PredictedTest)
```

```
round(defaultSummary(NN_ModelTest1),4)
```

```
##      RMSE  Rsquared      MAE
## 207116.55      0.71 138049.21
```

Correlations close to one indicate strong linear relationships between two variables. Therefore, the correlation here of about 0.845 indicates a fairly strong relationship. This implies that our model is doing a fairly good job, even with only a single hidden node

To see model performance metrics on the TestData sample, you can use a defaultSummary() function that, given two numeric vectors of data, calculates the mean squared error (MSE), the mean absolute error (MAE), and R-squared.

```
nn_model_results <- neuralnet::compute(nn_hs_model, hstest_norm[-1])
predicted_hs <- nn_model_results$net.result
```

```
cor(predicted_hs, hstest_norm$houseprice)
```

```
##           [,1]
## [1,] 0.8450572
```

```
pred_hs_org <- unnormalize(predicted_hs)
PredictedTest<-pred_hs_org
```

```
NN_ModelTest1<-data.frame(obs = hstest$houseprice, pred=PredictedTest)
```

```
round(defaultSummary(NN_ModelTest1),4)
```

```
##      RMSE  Rsquared      MAE
## 216038.8221      0.7141 140562.4617
```

Conclusion: - Looking at low R-Squared value (0.7141) and MAE (140562.4617) the NN model with single hidden layer seems to be a good fit here. - Lets see the impact of increasing the numbers of hidden layers

Second Model

As networks with more complex topologies are capable of learning more difficult concepts, let's see what happens when we increase the number of hidden nodes to five. We use the neuralnet() function as before, but add the parameter.

```
# hidden nodes = 2

nn_hs_model2 <- neuralnet(houseprice ~ ., data = hstrain_norm, hidden = 2)
plot(nn_hs_model2)

nn_model_results <- neuralnet::compute(nn_hs_model2, hstrain_norm[-1])
predicted_hs <- nn_model_results$net.result

cor(predicted_hs, hstrain_norm$houseprice)
```

```
##           [,1]
## [1,] 0.8485041
```

```
pred_hs_org <- unnormalize(predicted_hs)
PredictedTest<-pred_hs_org

NN_ModelTrain2<-data.frame(obs = hstrain$houseprice, pred=PredictedTest)

round(defaultSummary(NN_ModelTrain2),4)
```

```
##          RMSE  Rsquared          MAE
## 203963.05      0.72 136774.38
```

```
nn_model_results <- neuralnet::compute(nn_hs_model2, hstest_norm[-1])
predicted_hs <- nn_model_results$net.result

cor(predicted_hs, hstest_norm$houseprice)
```

```
##           [,1]
## [1,] 0.8556742
```

```
pred_hs_org <- unnormalize(predicted_hs)
PredictedTest<-pred_hs_org

NN_ModelTest2<-data.frame(obs = hstest$houseprice, pred=PredictedTest)

round(defaultSummary(NN_ModelTest2),4)
```

```
##          RMSE  Rsquared          MAE
## 210104.5055      0.7322 139851.4680
```

Conclusion: - Notice that the reported error (measured again by SSE) has been reduced by ~711 and the R-squared seems to be improved by ~2% i.e 0.732. We now obtain a correlation around 0.855, which is a improved a bit over the previous result of 0.845 with a single hidden node. - Overall, Neural Network seems to be improved with 2 neurons for given data set.

Third Model

Some insight about this third model which Prof. Dr. Hakan Gogtas thought us in the class is an activation function known as a rectifier has become extremely popular due to its success on complex tasks such as image recognition. A node in a neural network that uses the rectifier activation function is known as a rectified linear unit (ReLU). As depicted in the following figure, the rectifier activation function is defined such that it returns x if x is at least zero, and zero otherwise. The significance of this function is due to the fact that it is nonlinear yet has simple mathematical properties that make it both computationally inexpensive and highly efficient for gradient descent. Unfortunately, its derivative is undefined at $x = 0$ and therefore cannot be used with the `neuralnet()` function. Instead, we can use a smooth approximation of the ReLU known as softplus or SmoothReLU, an activation function defined as $\log(1 + e^x)$.

This activation function can be provided to `neuralnet()` using the `act.fct` parameter. Additionally, we add a second hidden layer of five nodes by supplying the hidden parameter the integer vector `c(1, 1)`. This creates a two-layer network, each having five nodes, all using the softplus activation function:

```
# hidden nodes = 2,2
nn_hs_model3 <- neuralnet(houseprice ~ ., data = hstrain_norm, hidden = c(1,1))
plot(nn_hs_model3)
```

```

nn_model_results <- neuralnet::compute(nn_hs_model3, hstrain_norm[-1])
predicted_hs <- nn_model_results$net.result

cor(predicted_hs, hstrain_norm$houseprice)

##           [,1]
## [1,] 0.8426366

pred_hs_org <- unnormalize(predicted_hs)
PredictedTest<-pred_hs_org

NN_ModelTrain3<-data.frame(obs = hstrain$houseprice, pred=PredictedTest)

round(defaultSummary(NN_ModelTrain3),4)

##      RMSE  Rsquared      MAE
## 207119.56      0.71 137983.31

nn_model_results <- neuralnet::compute(nn_hs_model3, hstest_norm[-1])
predicted_hs <- nn_model_results$net.result

cor(predicted_hs, hstest_norm$houseprice)

##           [,1]
## [1,] 0.8450095

pred_hs_org <- unnormalize(predicted_hs)
PredictedTest<-pred_hs_org

NN_ModelTest3<-data.frame(obs = hstest$houseprice, pred=PredictedTest)

round(defaultSummary(NN_ModelTest3),4)

##      RMSE  Rsquared      MAE
## 216072.944      0.714 140513.463

```

Conclusion: - Notice that the reported error (measured again by SSE) has been increased by ~662 and the R-squared seems decreased 0.2% i.e..0.714. we now obtain a correlation around 0.84 - Overall, Neural Network seems to be work better with 2 neurons for the given data set.

VI. Model Limitation and Assumptions

Following is a head-to-head comparison table among Neural Networks, LASSO, multiple linear regression to assess which model offers best performance.

Model	R^2	MAE
Neural Networks	0.732	543295.4
LASSO	0.6559796	138351.9
Linear Regression	0.6559345	138543.8

For MSE, Lasso model performs better. MSE NN: 3.213281e+14 MSE Lasso: 4.955058e+10 MSE WLS: 4.391894e+11

From data above, NN has the highest R^2 , and Lasso has the second highest R^2 . As a result, we pick NN as a Champion model, and Lasso as a benchmark model.

Model	Train R^2	Test R^2
Neural networks	0.71	0.732
LASSO	0.6555356	0.6559796
Regression Tree	0.6149	0.5902
WLS	0.6405230	0.6525047
Linear	0.655614	0.6559345

Model Assumptions

So far, we build parametric regression models and non-parametric regression models to seek the model that predicts house price with the highest coefficient of determination R^2 and lowest mean squared error (MSE). Following table summarizes the assumptions and limitations of the models we build so far.

Model	Assumptions	Limitation
Neural Networks	no distribution assumption	over-fitting, hidden layers are blackbox, slow process
LASSO	normality and constant variance of errors; linearity and independence	biased estimate of coefficient; arbitrary variable selection
Multiple Linear	Normality and constant variance of errors; linearity and independence	multicollinearity

Since NN is a non-parametric model, we are much more relaxed in terms of assumption. However, the hidden layers are blackbox which cause issues of overfitting and relatively longer processing time.

The assumption for LASSO is the dependent and independent variables are linearly related. LASSO is the best in fixing the issues of multicollinearity which is confirmed by the large VIF numbers. One of the main limitations of LASSO regression is that the coefficients that are produced by a LASSO model are biased. The L1 penalty that is added to the model artificially shrinks the coefficients closer to zero, or in some cases, all the way to zero. Since the coefficient estimates in a LASSO model are biased, it is difficult to estimate accurate standard errors for them. Another downside of LASSO models is that they tends to be unstable when trained on data with correlated features.

VII. Ongoing Model Monitoring Plan

1. Reassess the model's performance using 'Rsquared' and 'Adjusted Rsquared' and "MAE" and make necessary adjustments over time
2. Check variables included in the model are significant at certain level.
3. Ensuring data set is of high quality, and the variables are consistently measured over the long term.
4. Ensure data remains uniform and coherent overtime.
5. Focus on whether data adheres to predefined rules, constraints, or acceptable value ranges. Invalid data, such as unrealistic values or outliers, can skew analysis results, emphasizing the need for thorough validation.
6. Periodically conduct out-of-sample testing by using a portion of the data not seen during model development.
7. Document the model development process, including assumptions, methodologies, and any decisions made during the modeling process, detailed records of data transformations and feature engineering steps will be helpful in long term maintenance.

VIII. Conclusion

In conclusion, we were able to determine that the NN model is the most successful and accurate for predicting house prices. Linear regression and LASSO are very close together as benchmark models. All of the variables from the dataset are significant. From this, we find that house prices can be successfully predicted in the area of data. This could be useful to future real estate investors and current homeowners alike. Our dataset was limited to King County, Washington. We would require a broader dataset to expand these predictions across the United States or other countries.

Bibliography

- Chwiałkowski, C.; Zydron, A.; Kayzer, D. 2022. *Assessing the Impact of Selected Attributes on Dwelling Prices Using Ordinary Least Squares Regression and Geographically Weighted Regression: A Case Study in Poznan, Poland*. Land. <https://doi.org/10.3390/land12010125>.
- GeeksforGeeks. 2008. *R Programing for Data Science*. GeeksforGeeks. <https://www.geeksforgeeks.org/r-programming-for-data-science/?ref=lbp>.
- Geerts, M.; vanden Broucke, S.; De Weerd, J. 2023. *A Survey of Methods and Input Data Types for House Price Prediction*. ISPRS Int. J.Geo-Inf. <https://doi.org/10.3390/ijgi12050200>.
- MIRANDA CRACE. 2023. *How to Calculate a Home's Price Per Square Foot*. ROCKET Mortgage. <https://www.rocketmortgage.com/learn/price-per-square-foot#:~:text=Price%20per%20square%20foot%20is%20a%20metric%20used%20in%20the%20upgrades%20and%20renovations%20and%20more>.

Appendix

A summary table of descriptive statistics of key variables

```
summarytools::descr(hs_keyvars)
```

```
## Descriptive Statistics
## hs_keyvars
## N: 21613
##
##          age_renovated  age_sold  bathrooms  bedrooms  condition  floors
## -----
##          Mean          42.54    43.32         2.11      3.37         3.41      1.49
##          Std.Dev        28.82    29.37         0.77      0.93         0.65      0.54
##          Min            0.00      0.00         0.00      0.00         1.00      1.00
##          Q1             17.00    18.00         1.75      3.00         3.00      1.00
##          Median         39.00    40.00         2.25      3.00         3.00      1.50
##          Q3             62.00    63.00         2.50      4.00         4.00      2.00
##          Max           115.00   115.00         8.00     33.00         5.00      3.50
##          MAD            32.62    34.10         0.74      1.48         0.00      0.74
##          IQR            45.00    45.00         0.75      1.00         1.00      1.00
##          CV              0.68      0.68         0.36      0.28         0.19      0.36
##          Skewness        0.49      0.47         0.51      1.97         1.03      0.62
##          SE.Skewness      0.02      0.02         0.02      0.02         0.02      0.02
##          Kurtosis        -0.60    -0.66         1.28     49.05         0.53     -0.49
##          N.Valid        21613.00  21613.00    21613.00    21613.00    21613.00    21613.00
##          Pct.Valid       100.00    100.00     100.00     100.00     100.00     100.00
##
```

Table: Table continues below

```
##
##
##          grade  houseprice  houseprice_normlz  sqft_above  sqft_basement
## -----
##          Mean      7.66    540088.14          0.06    1788.39      291.51
##          Std.Dev    1.18    367127.20          0.05     828.09     442.58
##          Min        1.00     75000.00          0.00     290.00         0.00
##          Q1         7.00    321950.00          0.03    1190.00         0.00
##          Median     7.00    450000.00          0.05    1560.00         0.00
##          Q3         8.00    645000.00          0.07    2210.00        560.00
##          Max        13.00   7700000.00          1.00    9410.00     4820.00
##
```

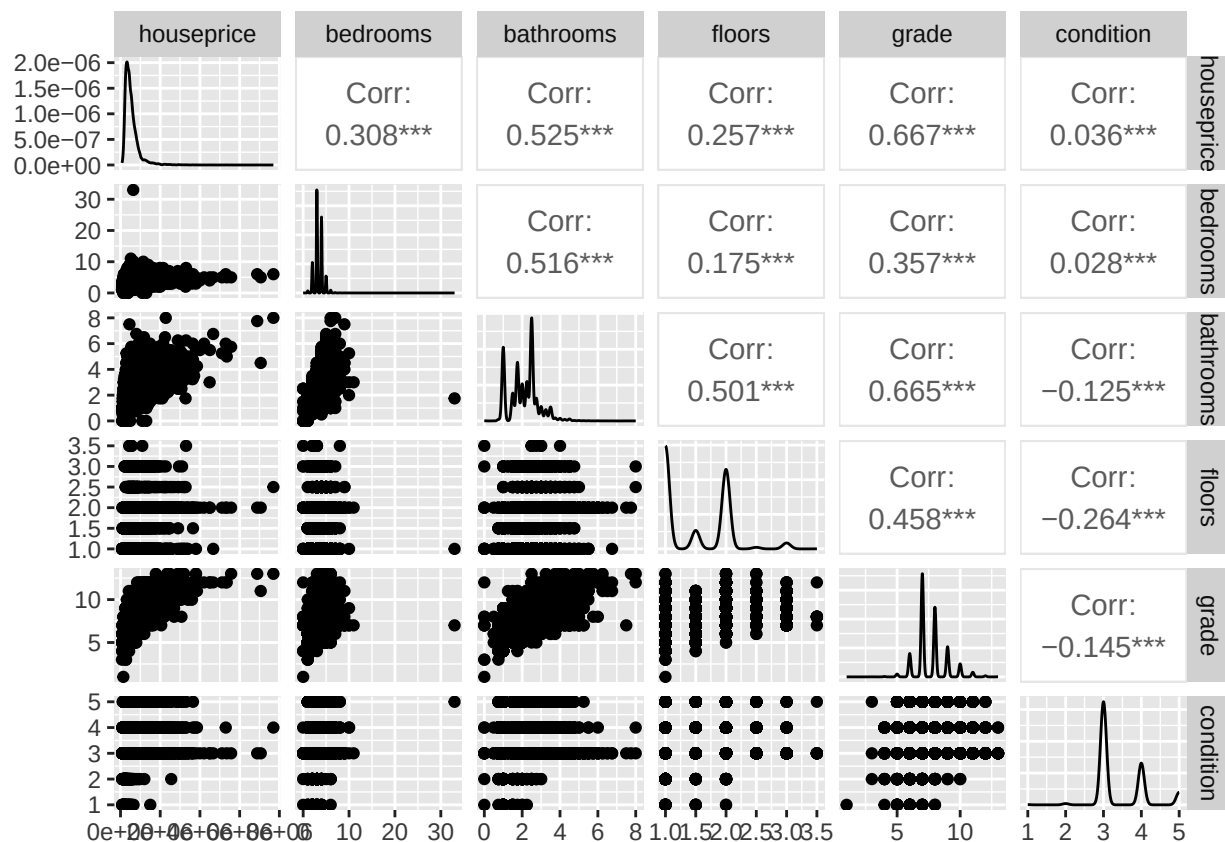
```
##          MAD      1.48    222390.00          0.03    667.17          0.00
##          IQR      1.00    323050.00          0.04    1020.00        560.00
##          CV       0.15      0.68          0.79      0.46          1.52
##      Skewness     0.77      4.02          4.02      1.45          1.58
##    SE.Skewness     0.02      0.02          0.02      0.02          0.02
##      Kurtosis     1.19     34.57         34.57      3.40          2.71
##      N.Valid    21613.00    21613.00    21613.00    21613.00    21613.00
##      Pct.Valid     100.00     100.00     100.00     100.00     100.00
##
```

```
## Table: Table continues below
##
```

```
##
##
##          sqft_living15    sqft_lot    sqft_lot15          view    waterfront
## -----
##      Mean          1986.55    15106.97    12768.46        0.23          0.01
##    Std.Dev          685.39    41420.51    27304.18        0.77          0.09
##      Min           399.00     520.00     651.00        0.00          0.00
##      Q1          1490.00     5040.00     5100.00        0.00          0.00
##    Median          1840.00     7618.00     7620.00        0.00          0.00
##      Q3          2360.00    10688.00    10083.00        0.00          0.00
##      Max          6210.00   1651359.00    871200.00        4.00          1.00
##      MAD           607.87     3881.45     3713.91        0.00          0.00
##      IQR           870.00     5648.00     4983.00        0.00          0.00
##      CV            0.35       2.74       2.14        3.27         11.47
##    Skewness         1.11      13.06       9.51        3.40         11.38
##    SE.Skewness         0.02       0.02       0.02        0.02          0.02
##      Kurtosis         1.60     284.98     150.71       10.89        127.59
##      N.Valid    21613.00    21613.00    21613.00    21613.00    21613.00
##      Pct.Valid     100.00     100.00     100.00     100.00     100.00
##
```

Scatter plots of pairwise variables between response variable (houseprice) and other predictors by the feature of predictors.

```
GGally::ggpairs(hs_keyvars, columns = c(1, 8:12))
```



```
GGally::ggpairs(hs_keyvars, columns = c(1,15:16))
```

